

Simple As Possible-1

using combinational and sequential circuits



Course Code: EEE 4308

Course Name: Digital Electronics Lab

Project Name: SAP-1 Architecture-based 8-bit Computer Design and Implementation

Supervisor Name: Mr. Ahmad Shafiullah

Group Name: Twisted Transistor

Members:

1. Feehad Kamal

ID: 200021218

2. Shafayet Sadik Sowad

ID: 200021248

3. M.M. Sadman Shafi

ID: 200021240

4. Rahib Bin Hossain

ID: 200021241

5. Wahiduzzaman Rishad

ID: 200021228

Objective:

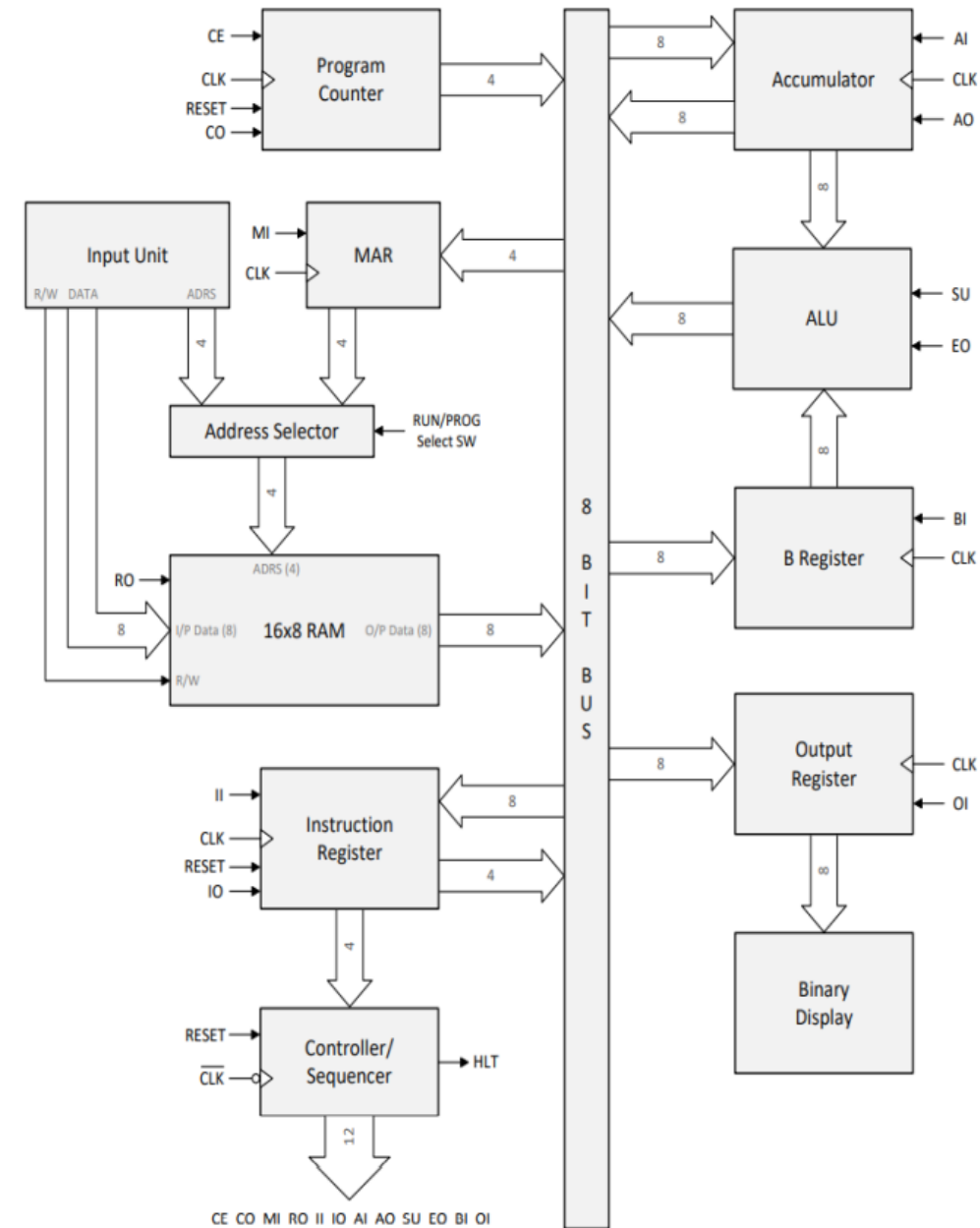
The objective of the project is to implement an 8-bit computer based on the architecture of SAP-1 (Simple-As-Possible) using combinational and sequential circuits.

Brief Description of SAP-1: SAP stands for Simple As Possible. The SAP-1 design contains the basic necessities for a functional Microprocessor. Its primary purpose is to develop a basic understanding of how a microprocessor works, interacts with memory and other parts of the system like input and output. The instruction set is very limited and is simple.

The computer must have the following features and functionalities:

1. A single 8-bit bus for address and data transfer.
2. 16x8 bit random access memory (RAM) for storing program and data.
3. A program counter for containing the address of the next instruction (should be capable of counting from 0000 to 1111).
4. Input unit for writing the program into RAM before running the computer.
5. Memory address register (MAR) for accessing RAM data.
6. An arithmetic and logic unit (ALU) capable of 8-bit addition and subtraction. Two registers Accumulator (register A) and register B (both 8-bit) are also needed for storing data for ALU operation.
7. A control unit for controlling all other blocks (registers, memory, ALU etc.)
8. Output unit capable of showing the output in binary form.
9. Main clock signal driving the computer should have two modes of operation- (i) Auto and (ii) Manual.
10. The computer must have all the blocks shown in project diagram (Fig. 1). Specifications of each block are described later.

Project Diagram:

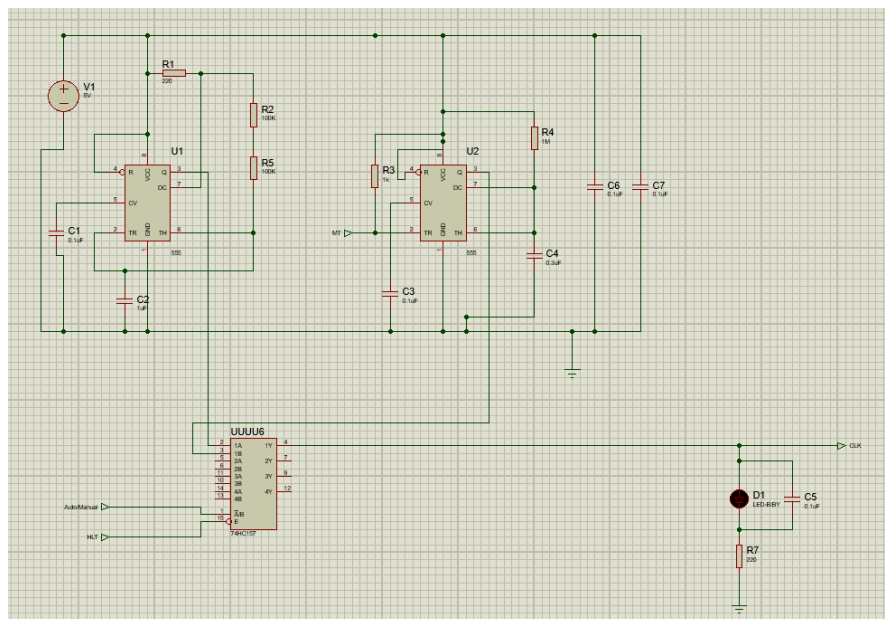
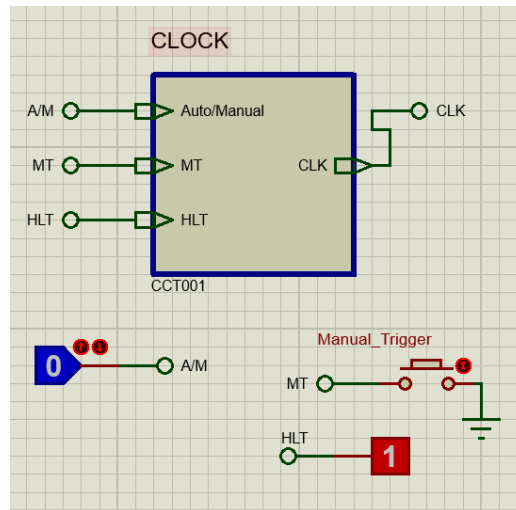


Proteus Simulations

Clock Pulse

A clock pulse is used to operate a Flip-flop. It is used to generate a constant pulse for Flip-Flop. The pulse goes from a low level 0 volt, the positive logical 0 condition, to a high level +5 volts, the positive logic logical 1 condition going between the two logic levels at a fixed frequency rate.

Software Simulation :



Here we have used three 555 timer. The 555 timer is connected with capacitors and resistors.

- The first part we have built is known as astable multivibrator. It will give us continuously 0 volt and +5 volts output respectively.
- Then we have built monostable multivibrator that is connected to a push switch when the switch is pushed and the circuit will be on there and it will give a stable +5 volts output. It is also known as manual pulse creator. When the output is positive it neutralize the bouncing effect of the astable multivibrator.
- Then we have built a bi-astable multivibrator. It is connected with a spdt switch which will give a stable input to the timer. When the switch will be on the it will give +5 volts output. This +5 volts neutralize the debouncing effect of the astable multivibrator.
- The output of this timers are connected with AND, OR and NOT gate. The final out of the combinational circuit is connected with a led. Observing the blink of the led we can get that our clock pulse is working.

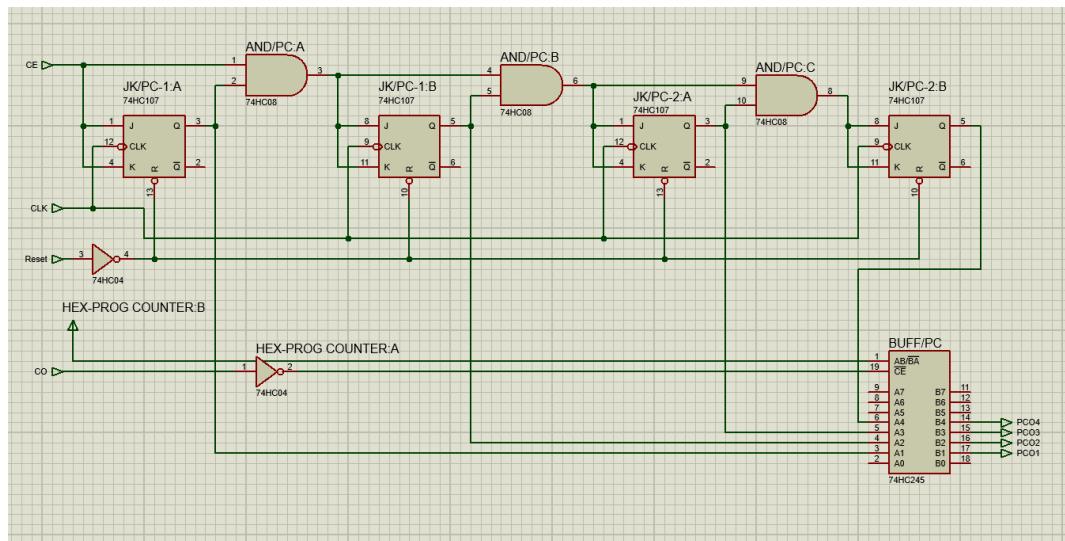
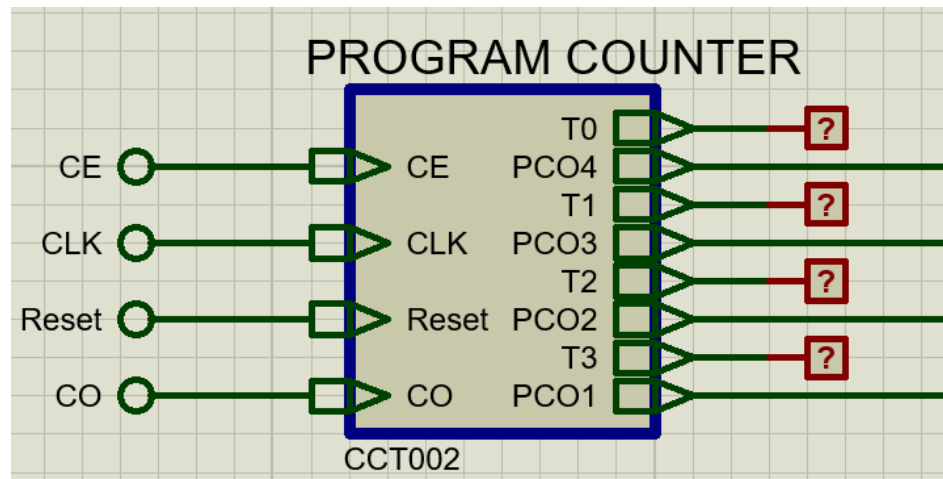
Challenges:

At first, after opening the proteus file if we set the switch for getting the astable output, then no output was obtained but after setting the switch for getting the monostable output once, and then setting the switch for the astable again, we obtained the astable output.

Program Counter

A program counter is a counter which can store and send out the memory address of the next instruction to be fetched. For our project, we used a 4-bit counter which can count 0000 to 1111.

Software Simulation:



We used 4 JK Flip-Flops to construct the Program Counter. Each Flip-Flop is in the toggle mode and the output of the first Flip-Flop provides as the clock pulse of the next Flip-Flop. We used RESET for each Flip-Flop. There is a CE (Counter Enable) button to control the pulse, in other words, to control the counter.

➤ We used a Tri State Buffer to maintain the bus contention. There is a CO (Counter Out) button to control the output of the buffer.

➤ We built a synchronous counter, since we focus to make our design much more simple and less expensive. Though there is a delay in this type of counter, which we encounter when we implement the circuit on breadboard.

Challenges:

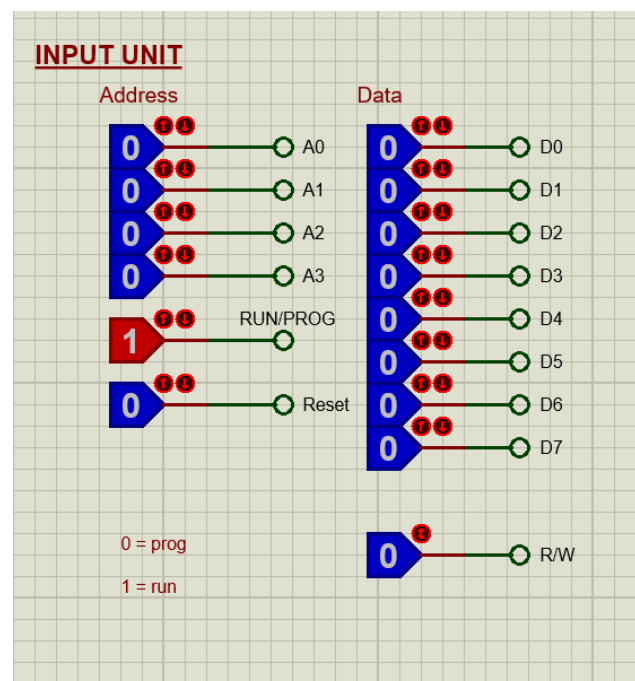
For program counter, we faced mainly the 'No reference, Partial Analysis error'. The reason behind it was we were using only 'Clock'. After that we used 'Dclock' from generator mode and then the problem was solved.

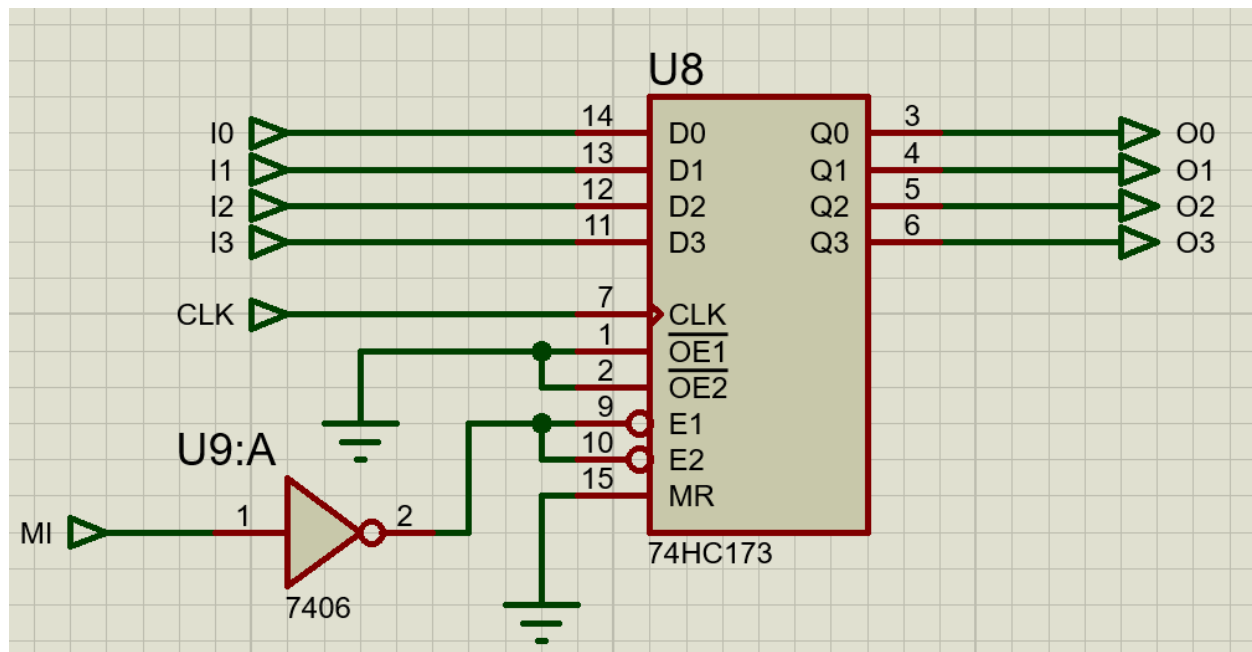
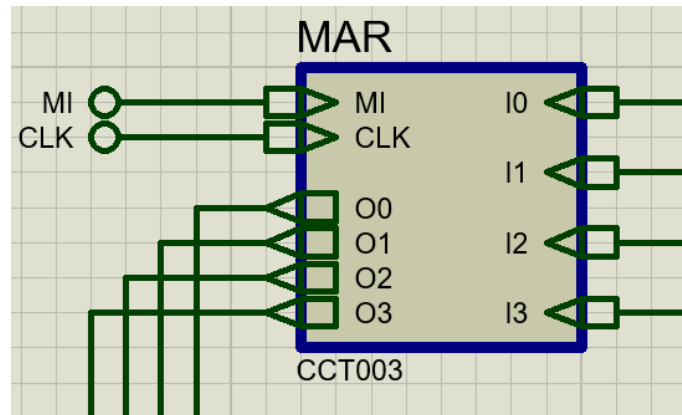
Input Unit and MAR:

MAR is a memory address register which stores the address of the data the RAM module should be pointing at. In general, MAR is a parallel load register that contains the next memory address to be manipulated. For example, the next address to be read or written.

Input Unit is used when the computer is needed to be programmed by writing data into the RAM in program state. There is a selector to choose between MAR output and input address output depending on the state (RUN/PROG).

Software Simulation:





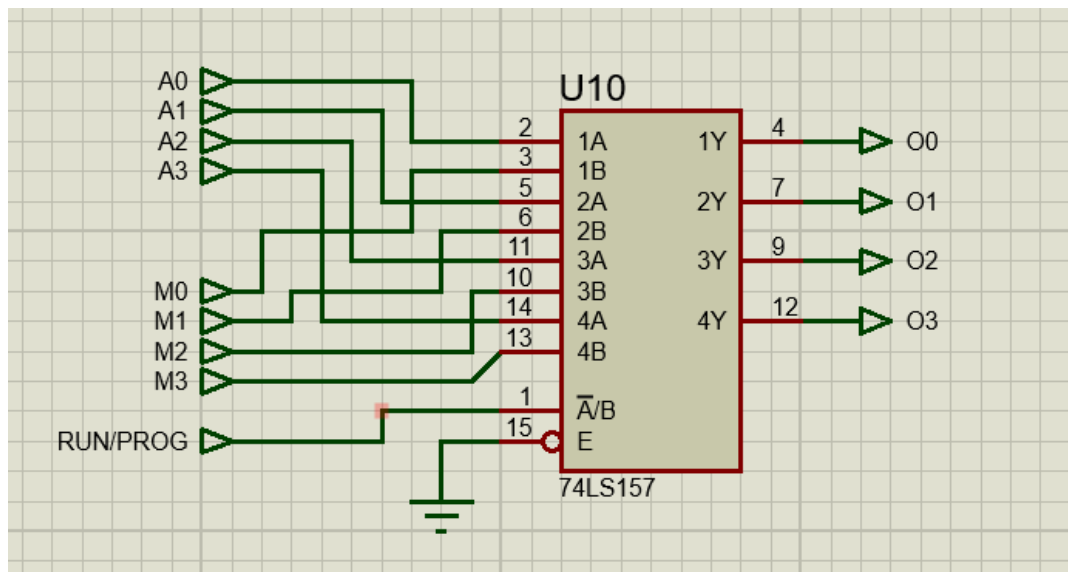
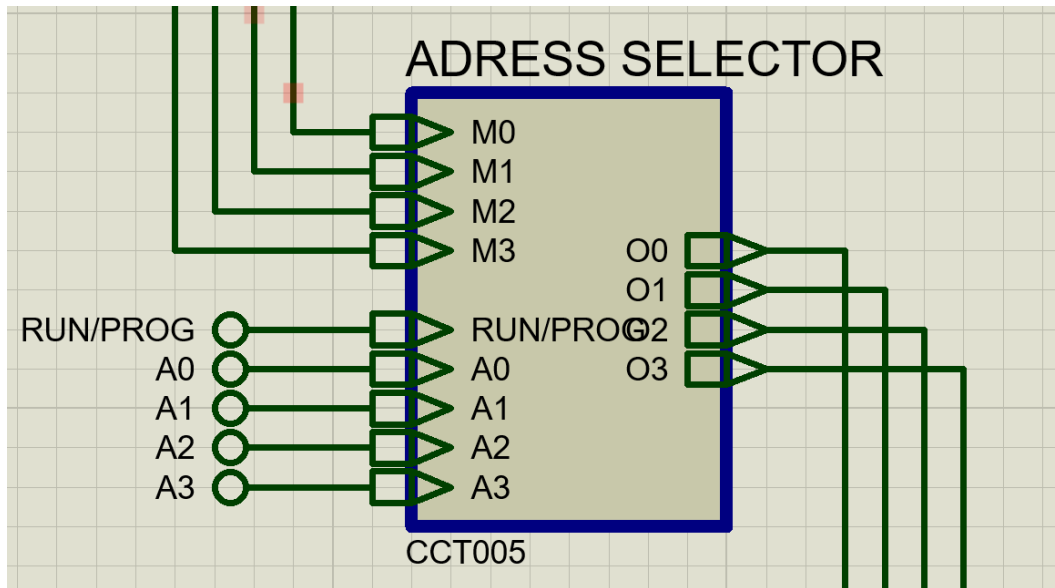
- In MAR we have used one 74HC273 IC . We have connected LEDs with the output pin of the IC . We have connected the output of the counter with the input pin of the IC which is used to implement MAR.
- We have used one AND gate to control the clock pulse with a switch.
- In input Unit we have taken all the input of the SAP using one 8-DIP switch, one 4-DIP switch and a push button switch.

Challenges:

We didn't face any problem in software simulation. Everything went smoothly for this part.

Address Selector

Software Simulation:

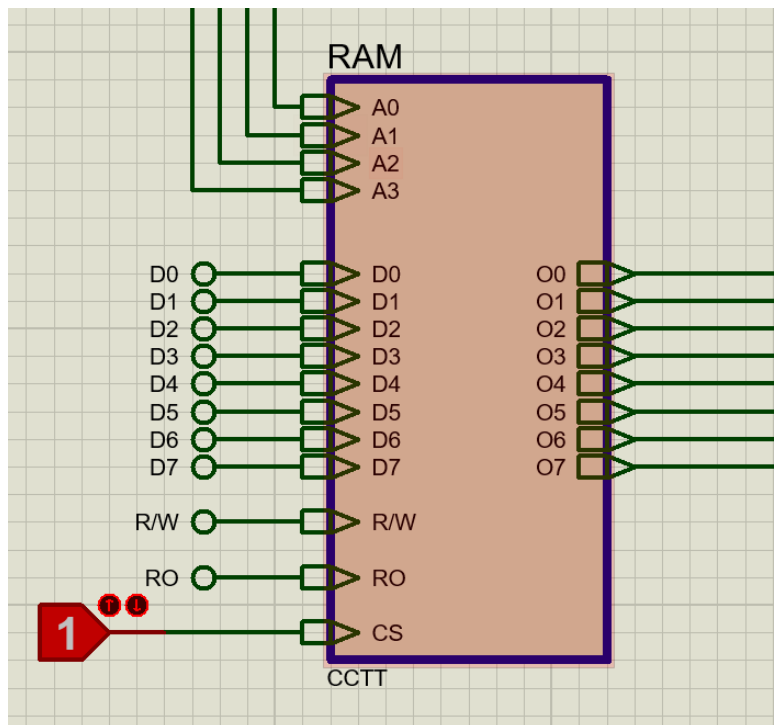


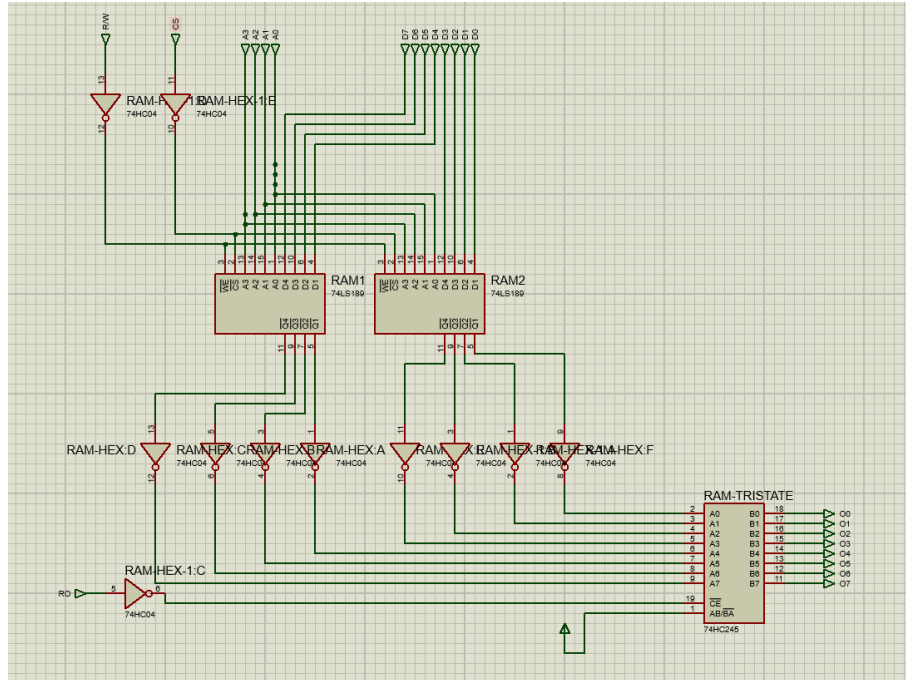
RAM:

Random Access Memory (RAM) size should be 16 x 8 bit (16 memory locations each storing 8 bits of data). The RAM will be programmed by means of the address switches and data switches of input unit. During a computer run, the RAM will receive its 4-bit address from the MAR.

Pin (Bit)	Type	Function
ADRS (4)	In	4-bit address input for accessing RAM locations.
I/P Data (8)	In	In PROG state, used for writing 8-bit data to addressed location.
R/W	In	Read or write mode.
CS	In	Chip select pin.
RO	In	RAM Out: Put the currently selected RAM byte onto the bus. Output is disconnected when RO is low.
O/P Data (8)	Out	8-bit output data.

Software Simulation:





- Here we have used two 74LS189 64-bit RAM.
- 8-bit data comes from the input unit directed by the Address Selector and stored in RAM.
- We have designed in such a way that 8-bit data is distributed equally in each of the RAM, so that each RAM has received 4-bit data input.
- Address of the data is also fixed. In our design, we have shorted the 4 address bits and provide input to each RAM.
- The output of the RAM is the complemented version of our desired output. So we have used invertors to get the proper output.
- Then, before pass the data to the bus, we have set a tri state buffer for controlling purpose.
- The read/write mode and enables of both RAM and Tri State Buffer ICs are connected accordingly.

Challenges:

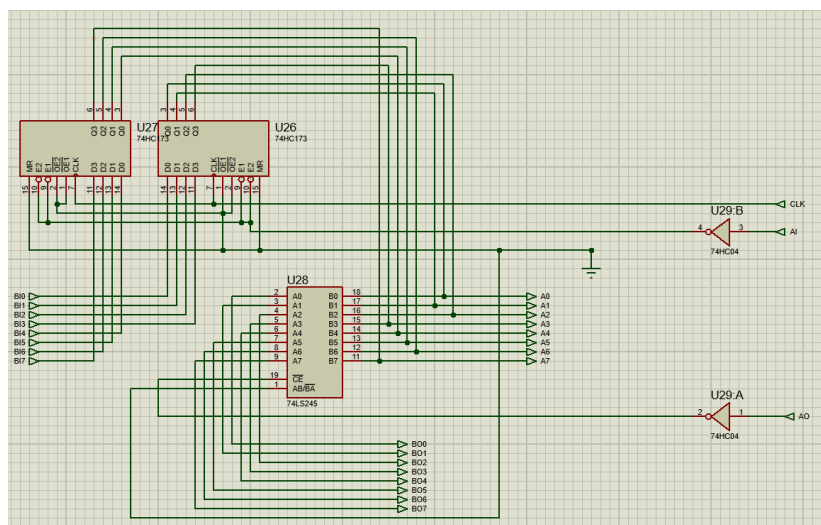
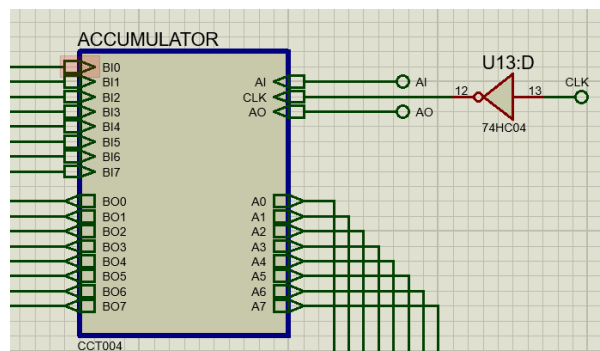
There wasn't any particular problem in the software simulation in Proteus. It went more or less smoothly.

Accumulator, B Register and Output Register:

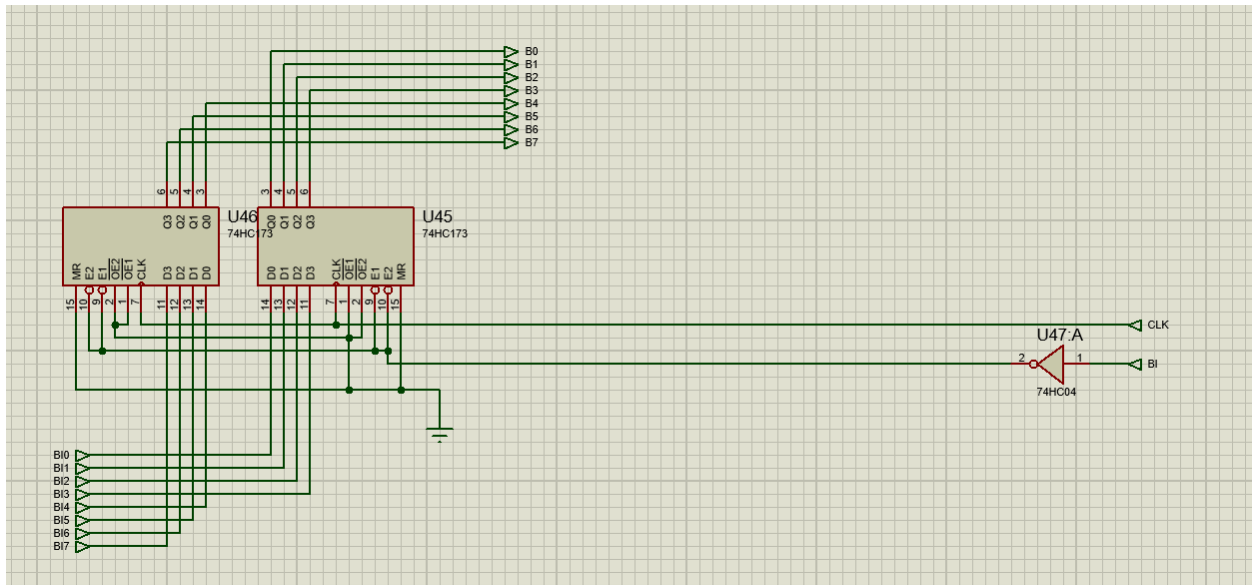
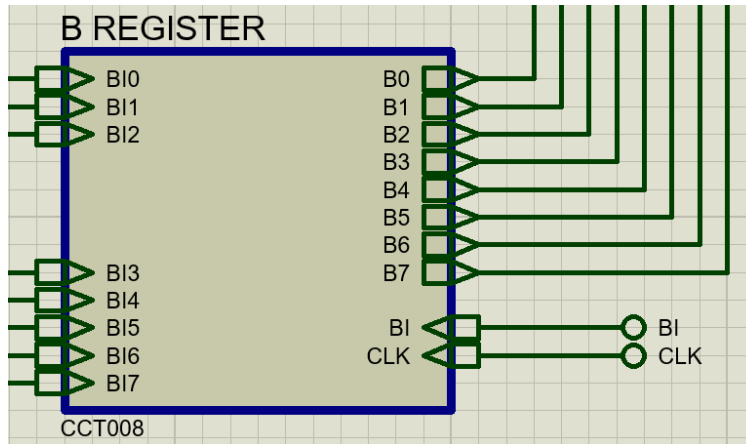
All of these registers will be of 8-bit size. Accumulator (also called as A register) should have the ability to output intermediate results after each ALU operation. Accumulator and B register will be used to store 8-bit data as input to ALU input. The purpose of output register is to store result to display when requested. Controller/sequencer unit sends control signals to these registers for controlling input/output enable. Description of these control bits.

Pin	Name	Function
AI	Accumulator In	Stores the current values of the bus into Accumulator.
AO	Accumulator Out	Accumulator sends its stored content to the bus. Output of accumulator is disconnected when AO is low.
BI	B Register In	Stores the current values of the bus into B Register.
OI	Output Register In	Stores the current values of the bus into Output Register.

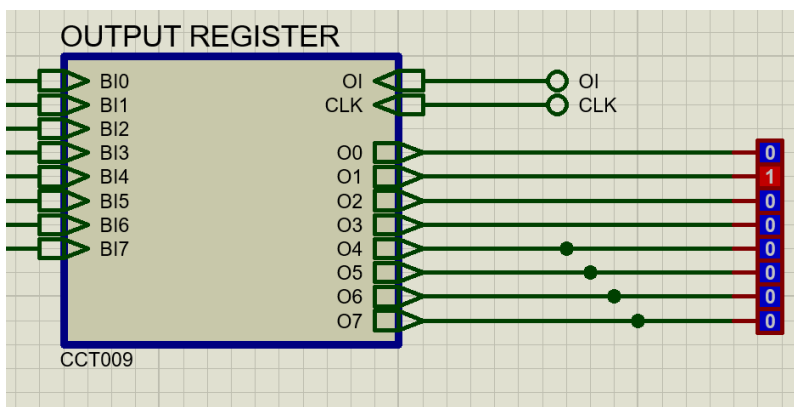
Software Simulation:

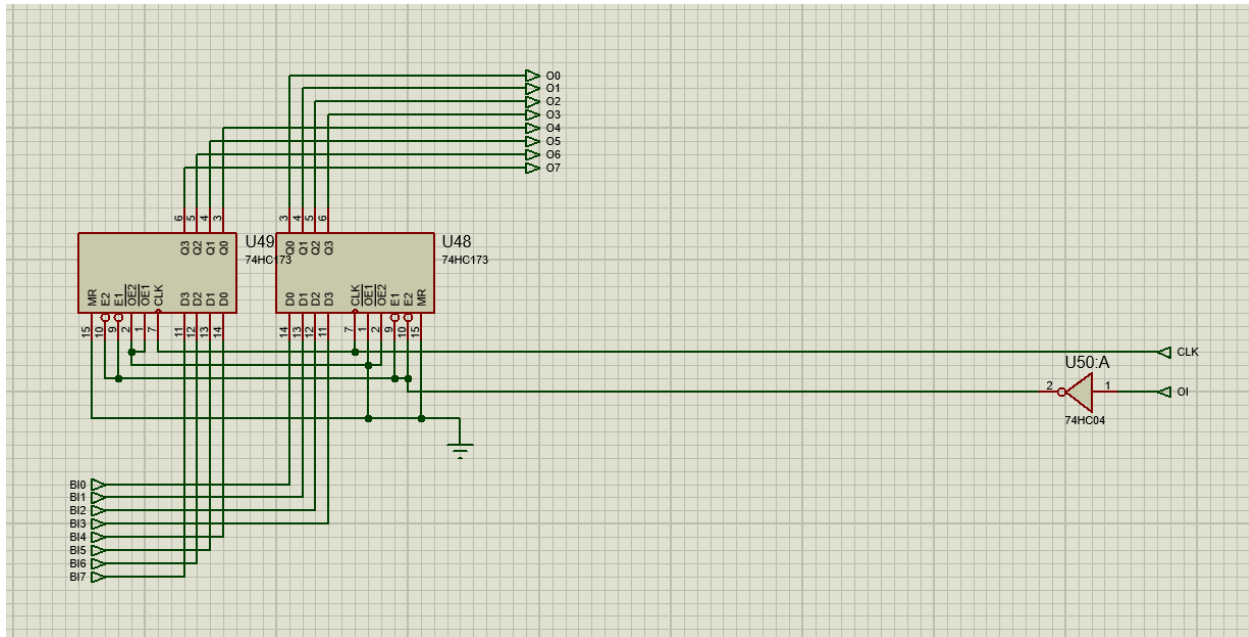


Accumulator



B Register





Output Register

- All registers have the same structure, where Accumulator both receives and delivers data to the bus, but B register only receives data from the bus. Both registers send the data to the ALU for different operations. The Output register stores and sends the data to the display for showing the result.
- For all the registers, we give 8-bit data input to the 8-bit register.
- We used Tri State Buffer to maintain bus contention.
- We connected the enable pin of the registers accordingly for controlling the registers.
- The output of the registers are also 8-bit data, which are sent to different modules (ALU, BUS, Display) according to our operation.

Challenges:

There wasn't any particular problem in the software simulation in Proteus. It went more or less smoothly.

Controller Sequencer

Controller-sequencer generates necessary control signals for each block so that actions occur in a desired sequence. 12 control bits come out of the controller-sequencer block. These control bits determine how all other blocks will react to the next positive CLK edge.

So, controller-sequencer must be designed in such a way so that correct control bits are already available to all the blocks before the next positive CLK edge.

12 control bits are –

CON = CE CO MI RO II IO AI AO SU EO BI OI

Functions of all 12 control bits are already described. Besides these 12 signals, controller-sequencer also generates a HLT (halt) signal, which can halt the process of computer by stopping the main clock.

We built our controller circuit with 5 T states. First 2 states are same for all the instructions, they vary for the next 3 states. The states are given below:

T1- CO, MI

T2- RO, II, CE.

T3-

ADD- IO, MI

SUB- IO, MI

LDA- IO, MI

OUT- AO, OI

T4-

ADD- RO, BI

SUB- RO, BI

LDA- AI, RO

OUT- ×

T5-

ADD- EO, AI

SUB- EO, AI, SU

LDA- ×

OUT- ×

We set the instructions in our own way.

LDA- 0000

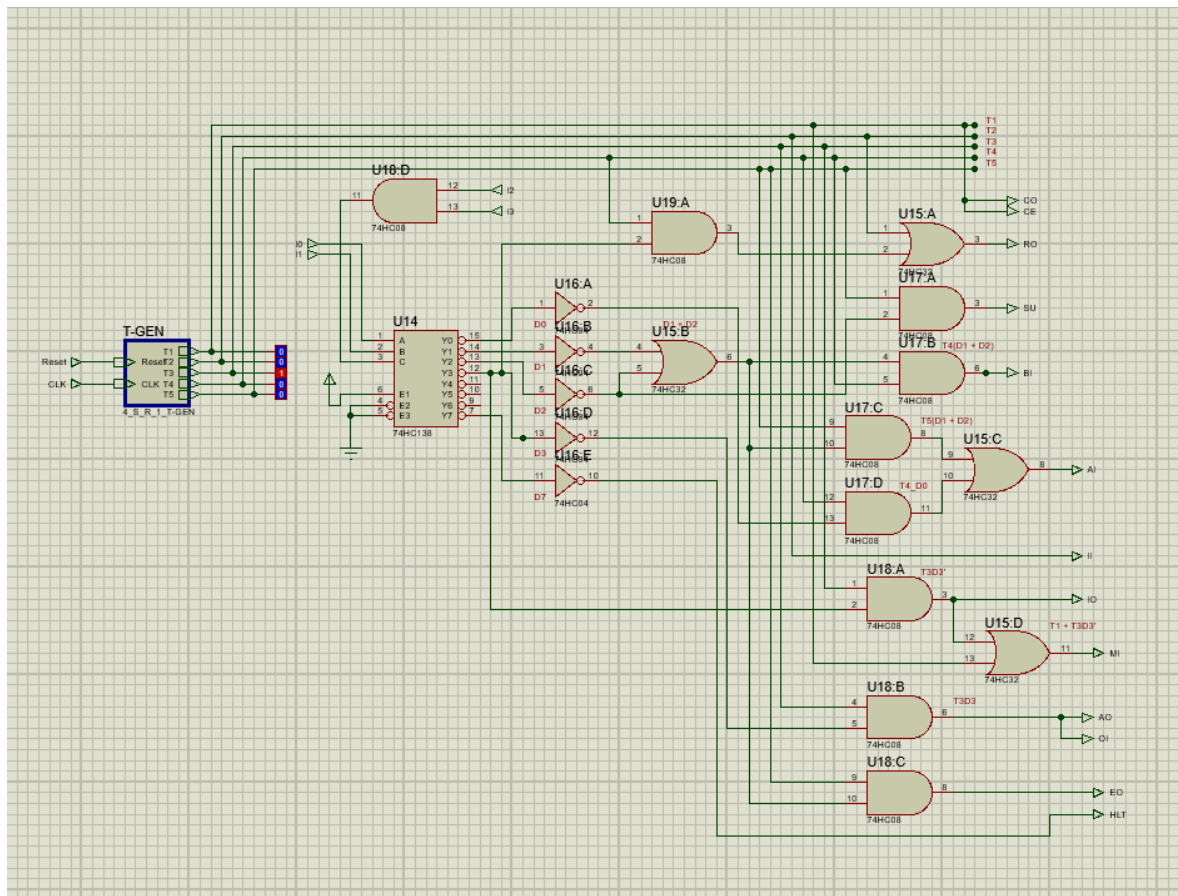
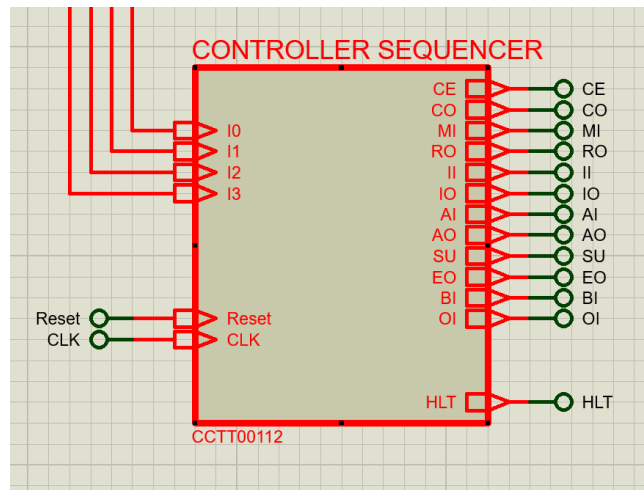
ADD- 0001

SUB- 0010

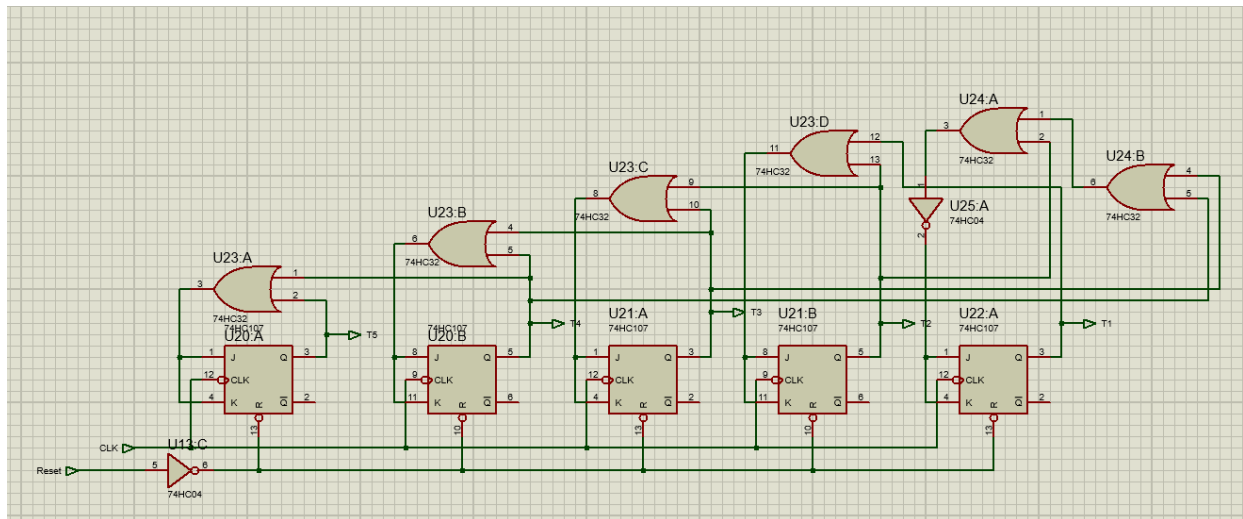
OUT- 0011

HLT- 1111

Software Simulation:



Sub Circuit Of Control Sequencer



T State Generator

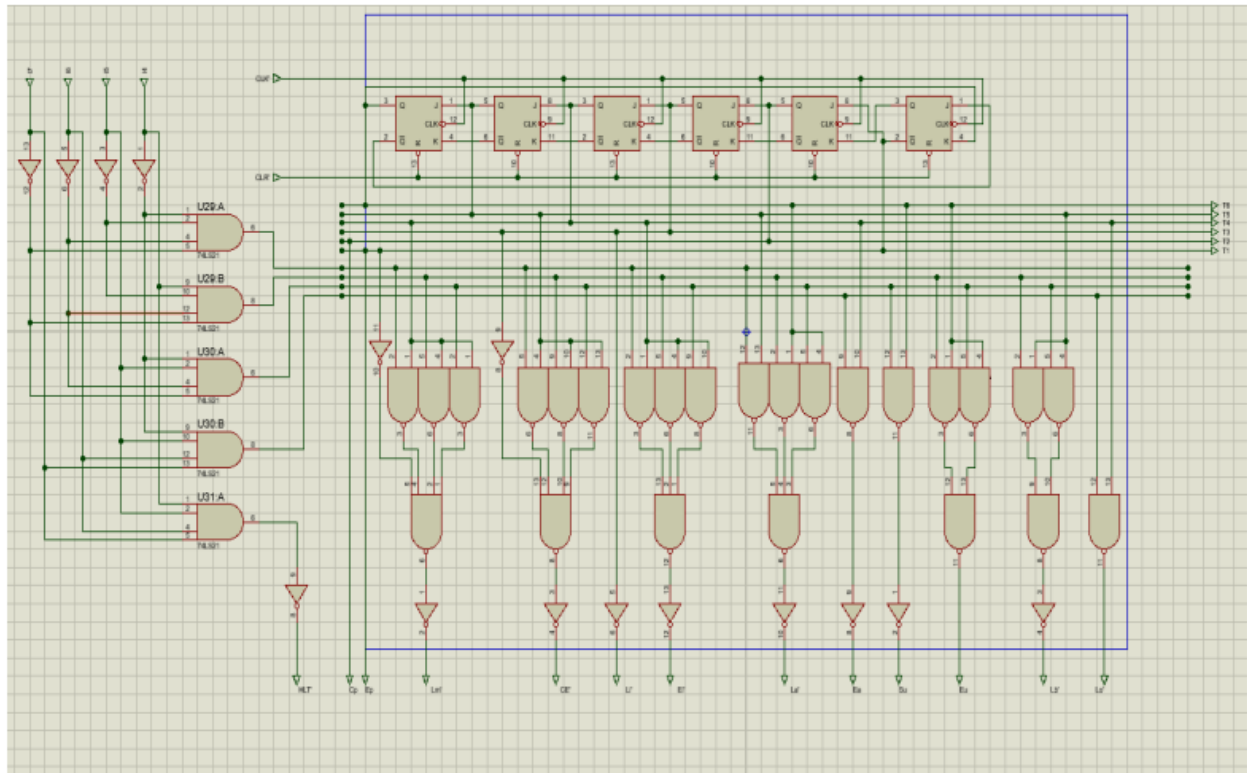
Challenges:

Since it is a huge module, we faced troubles building the entire circuit. Connections were very congested first, we had to make it decent.

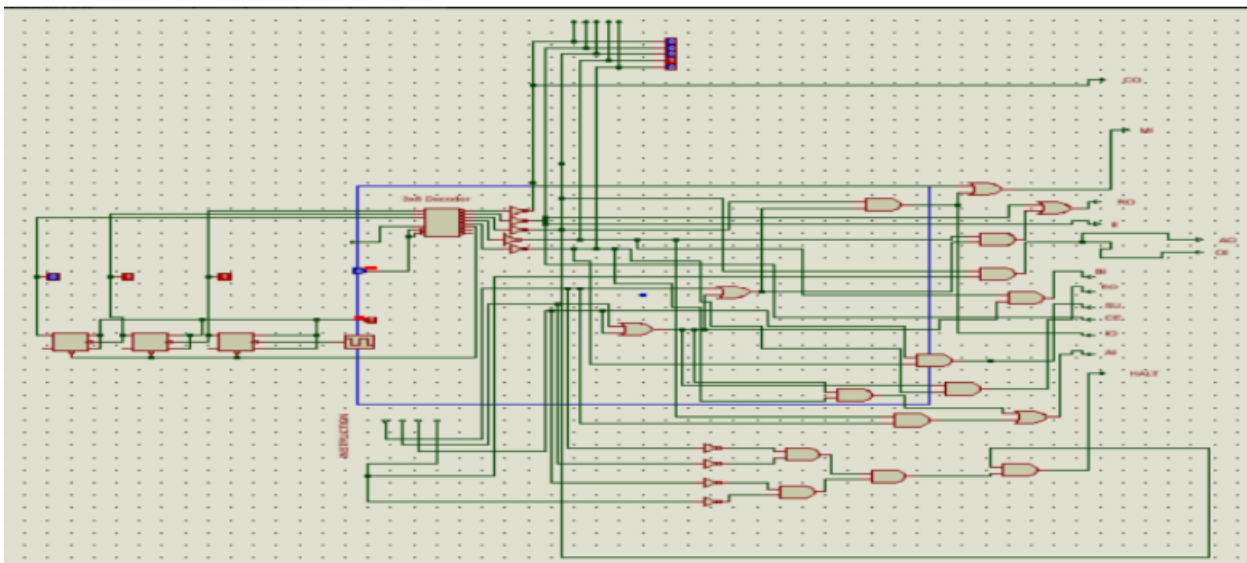
Importantly, we were asked to reduce the T states. We were successfully reduced the number of states from 6 to 5.

We made the controller with 6 states first, but that was too much clumsy. Then, we took many trial and error attempts and redesigned our controller

structure, and finally could make it a way with 5 T states. The circuit was simpler than the previous one and needed less gates.



The internal circuit for controller with 6 T states.



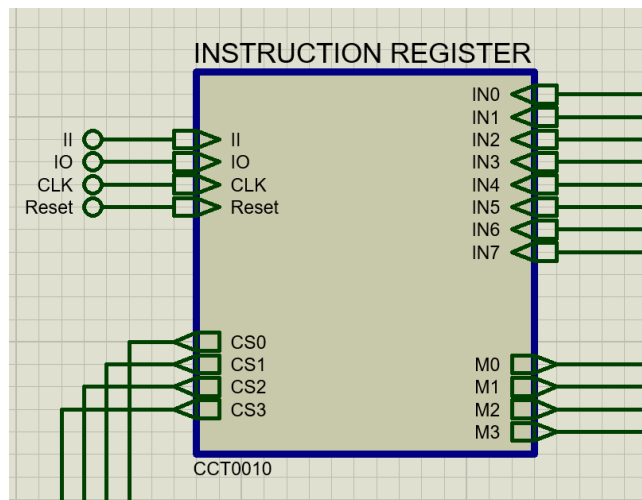
Internal circuit for controller with 5 T states

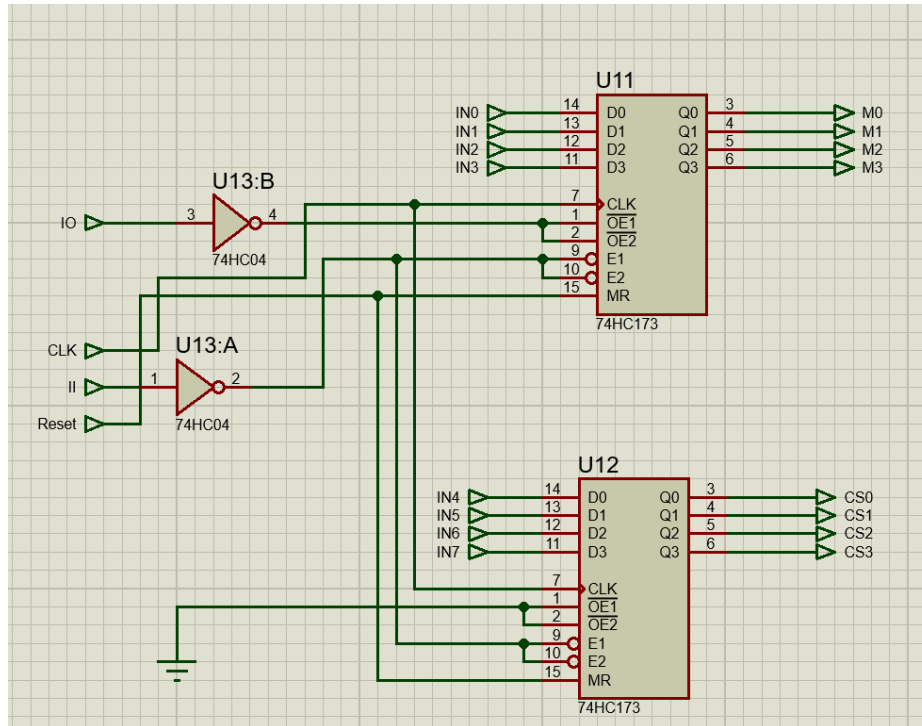
Instruction Registers

Instruction Register: Function of instruction register is to receive and store the 8-bit instruction placed on the bus from the RAM. The contents of the instruction register are then split into two nibbles (nibble means 4-bit). The upper nibble goes into the controller-sequencer while the lower nibble should be sent to the bus. A reset pin is required for resetting the instruction register, when computer starts. Control bits of instruction register are-

Pin	Name	Function
II	Instruction Register In	Writes the current values of the bus into the instruction register.
IO	Instruction Register Out	Sends the stored data (only lower nibble) onto the bus. Output pins are disconnected when AO is low.

Software Simulation:





Challenges:

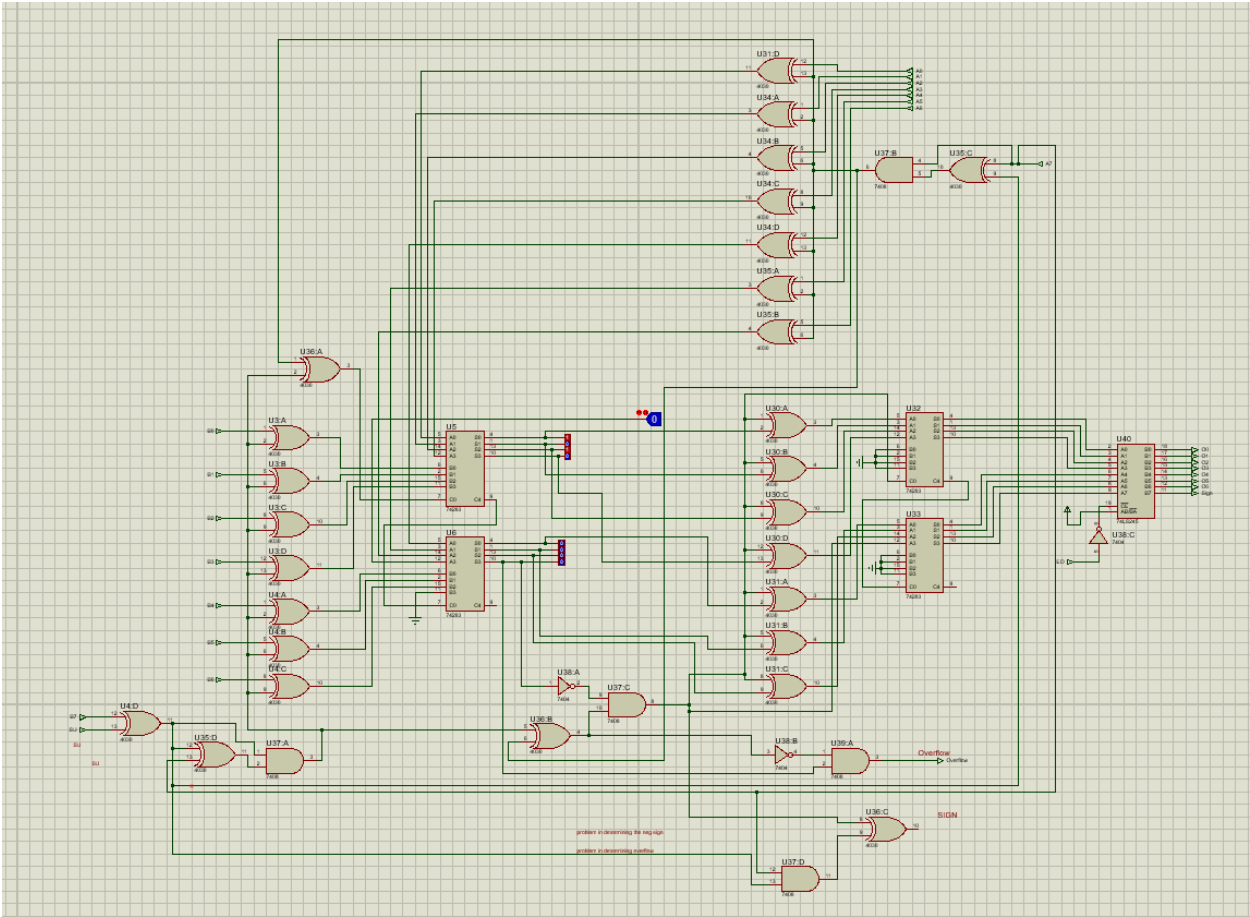
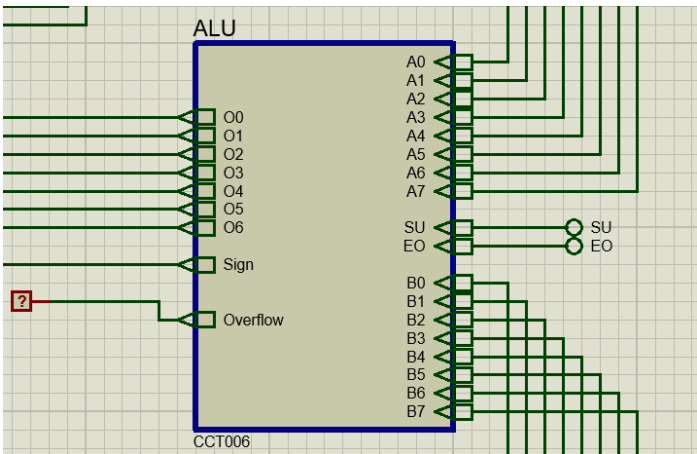
As we constructed other registers in the previous phase, so we made precautions for this phase beforehand and so we didn't face any major problems in building the instructions register circuit for this phase. All went smooth and decent.

Arithmetic and Logic Unit (ALU):

ALU should be capable of 8-bit addition and subtraction. For addition and subtraction, 2's complement approach should be used. Description of two control bits of ALU-

Pin	Name	Function
SU	Addition/Subtraction Select	When SU=0, addition operation, when SU=1, subtraction operation will be executed.
EO	ALU Out	Puts the ALU output onto the bus. Output of ALU should be disconnected when RO is low.

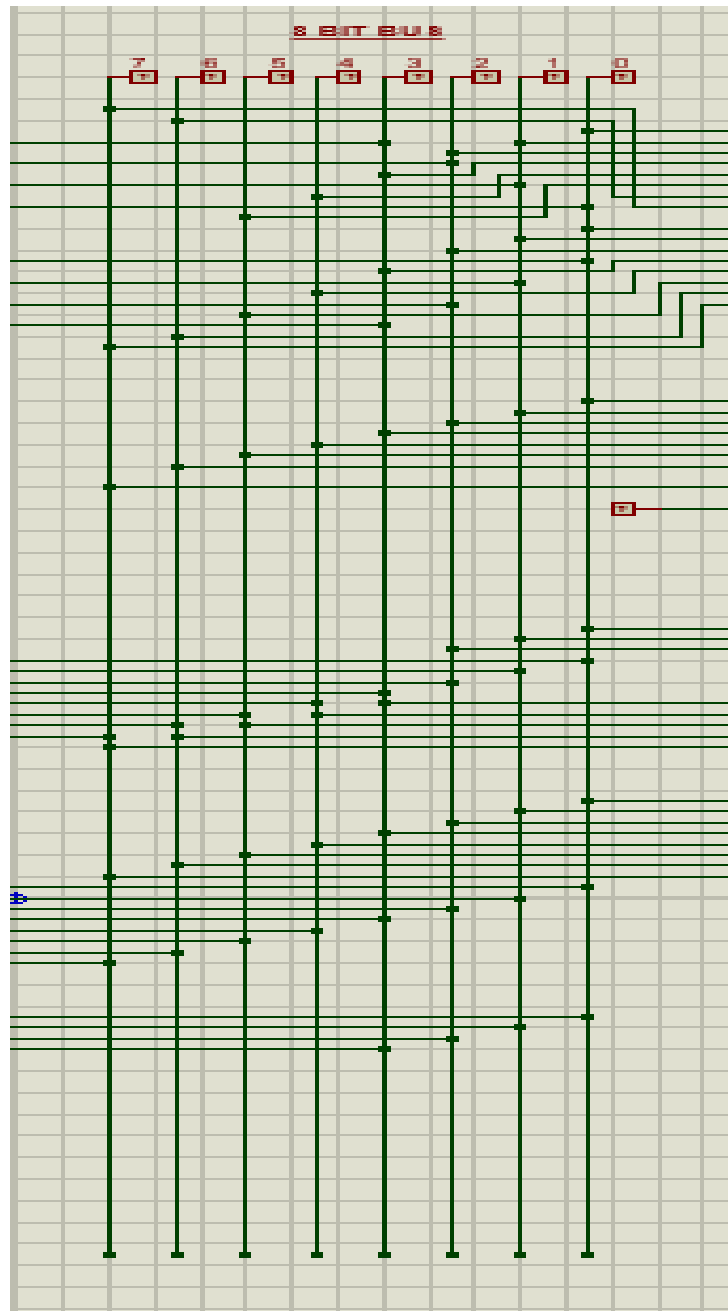
Software Simulation:



ALU Challenge:

Had to add a not gate with the output enable 'EO', so that the output enable becomes active high.

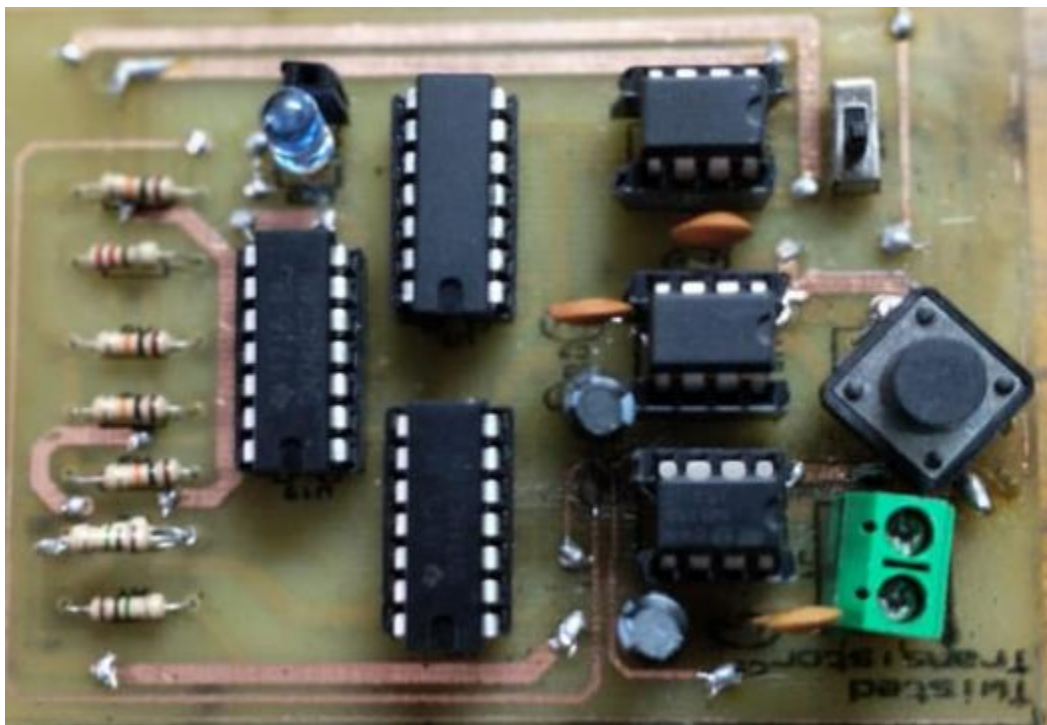
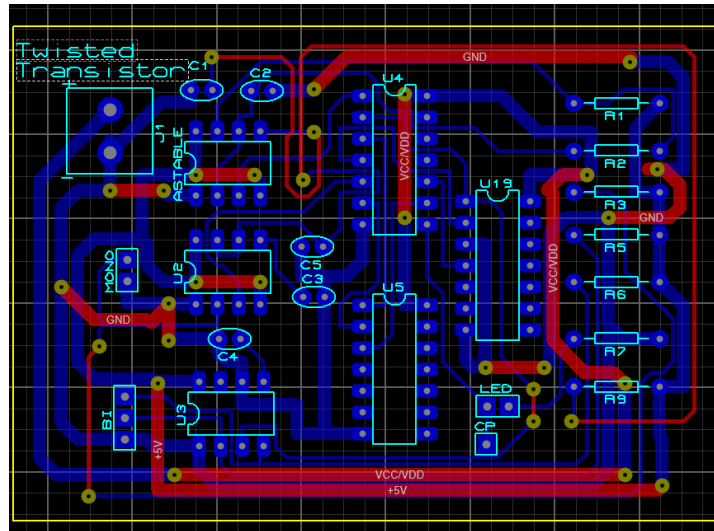
8-bit Bus Bar



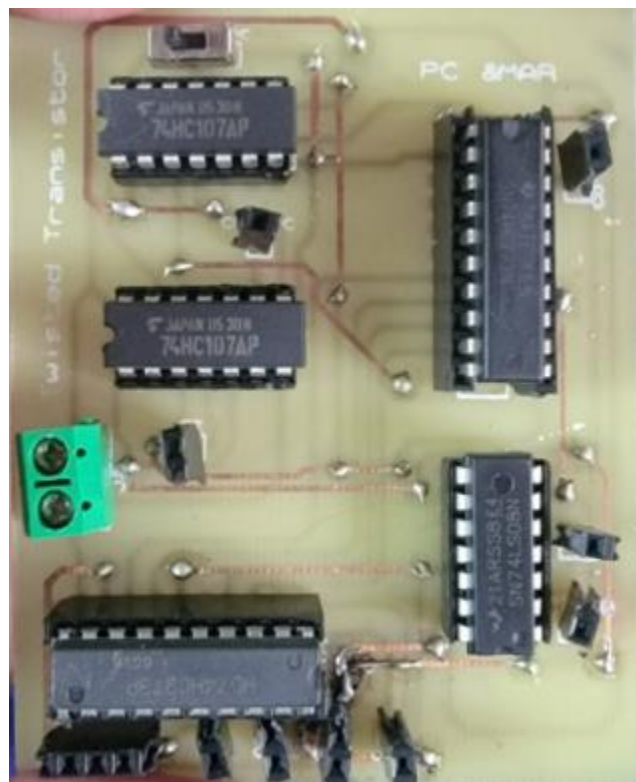
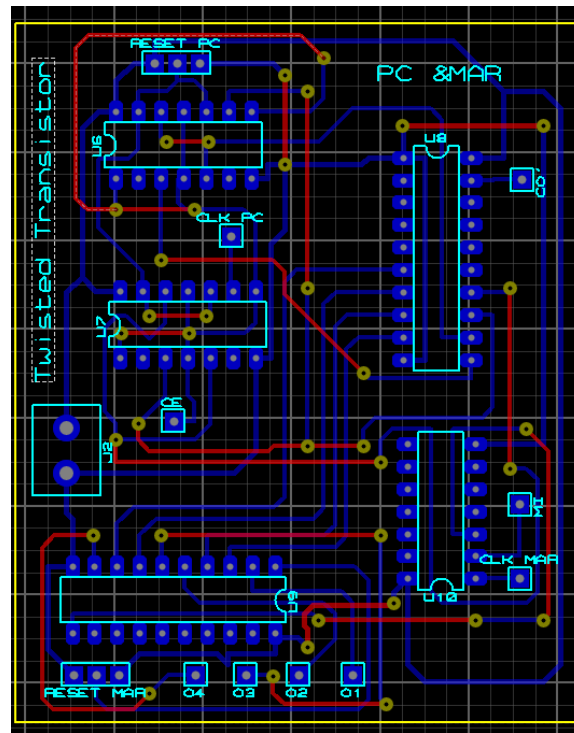
This bus bar is used for inter connecting all the modules, through which the data bits pass from one module to another.

PCB Design

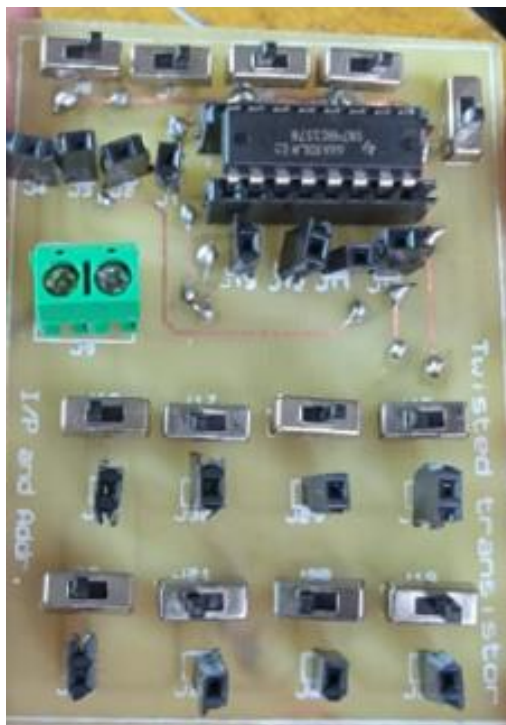
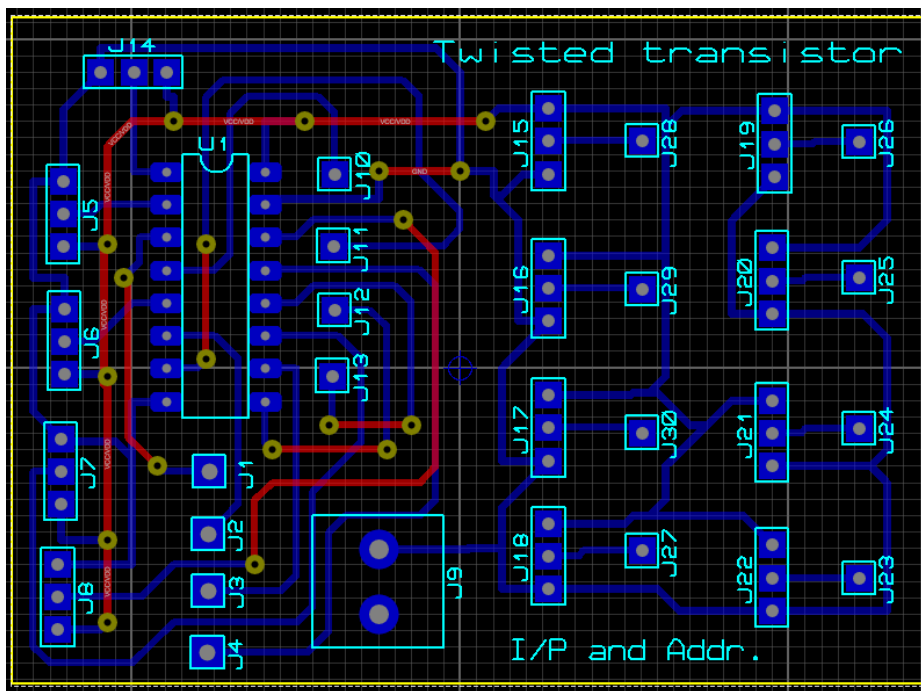
Clock Pulse



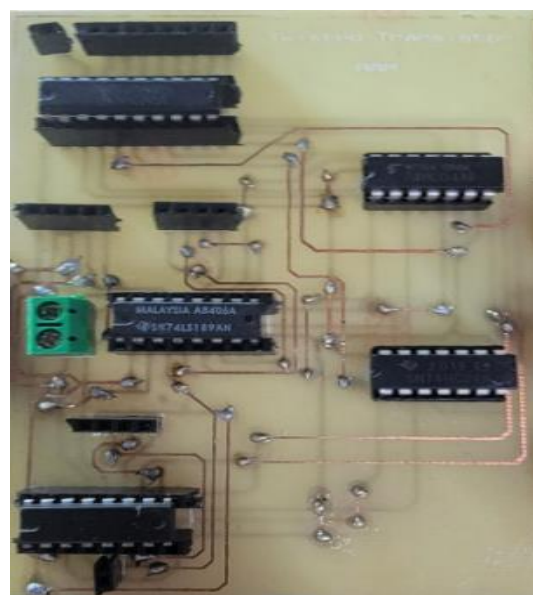
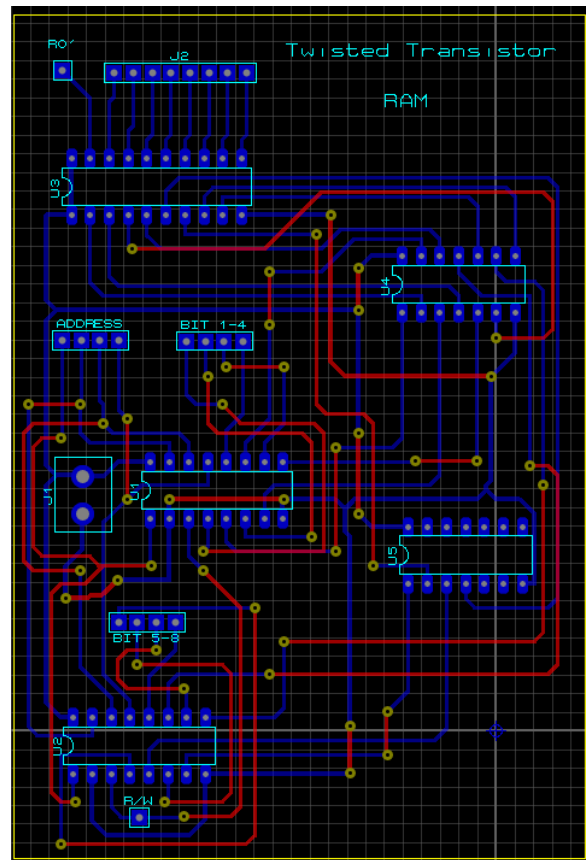
Program Counter & MAR



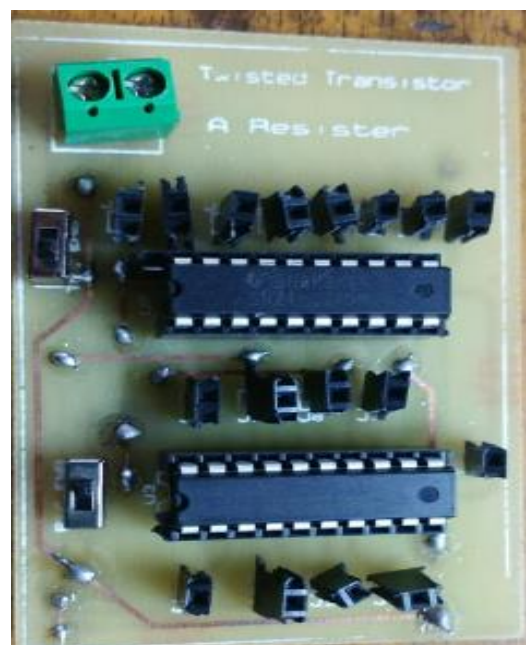
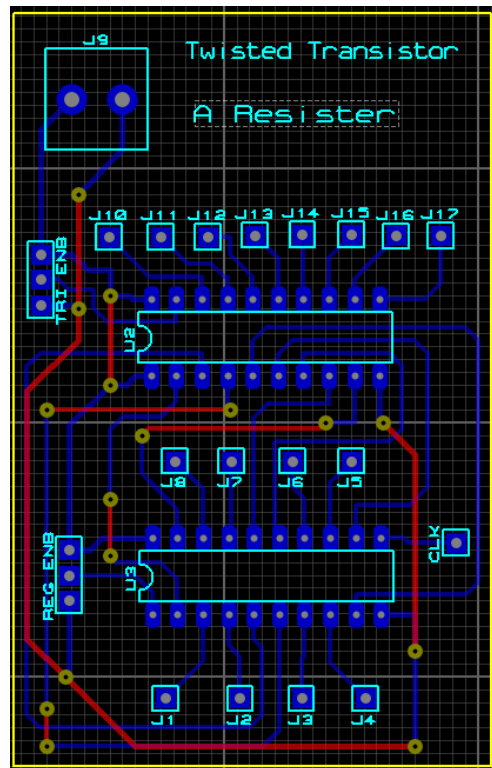
Input Unit & Address Selector



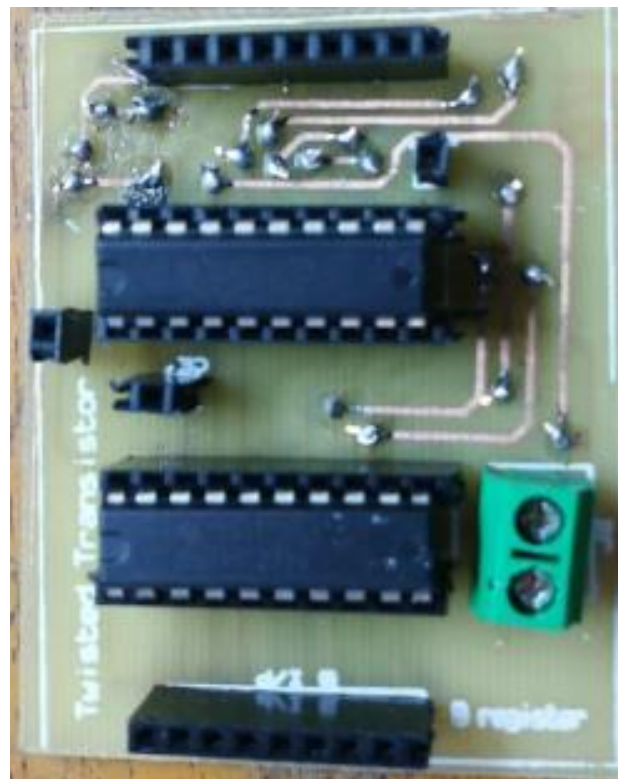
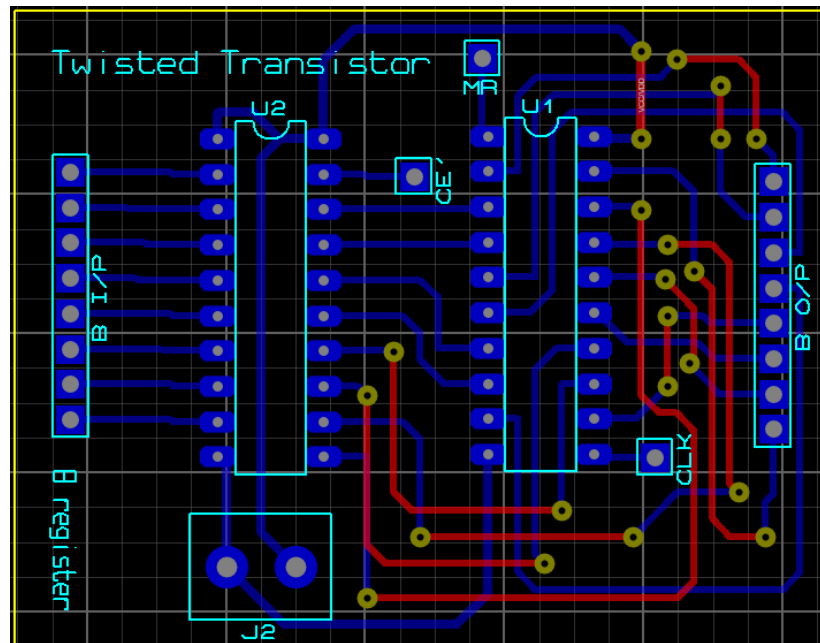
RAM



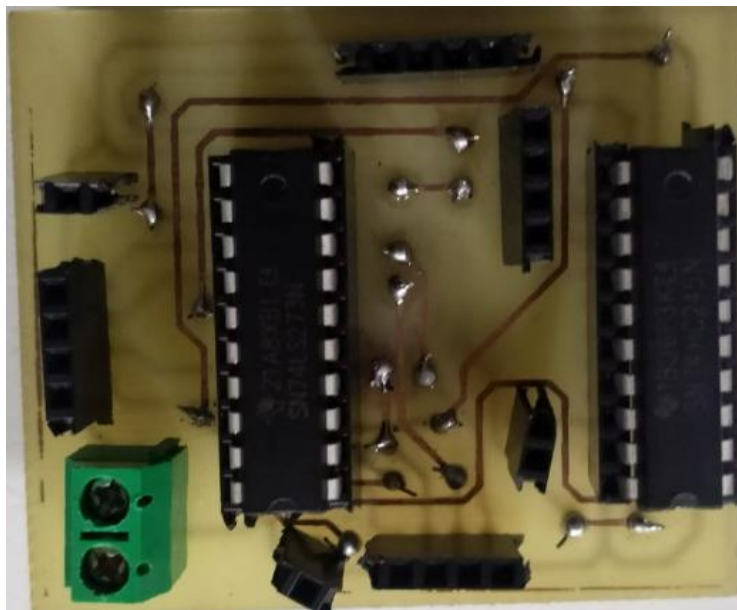
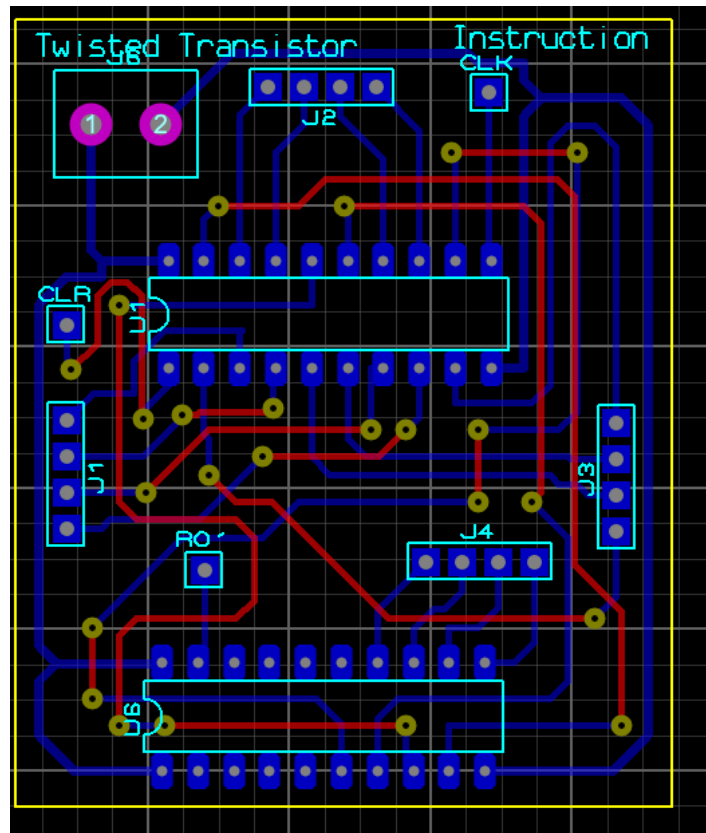
Accumulator



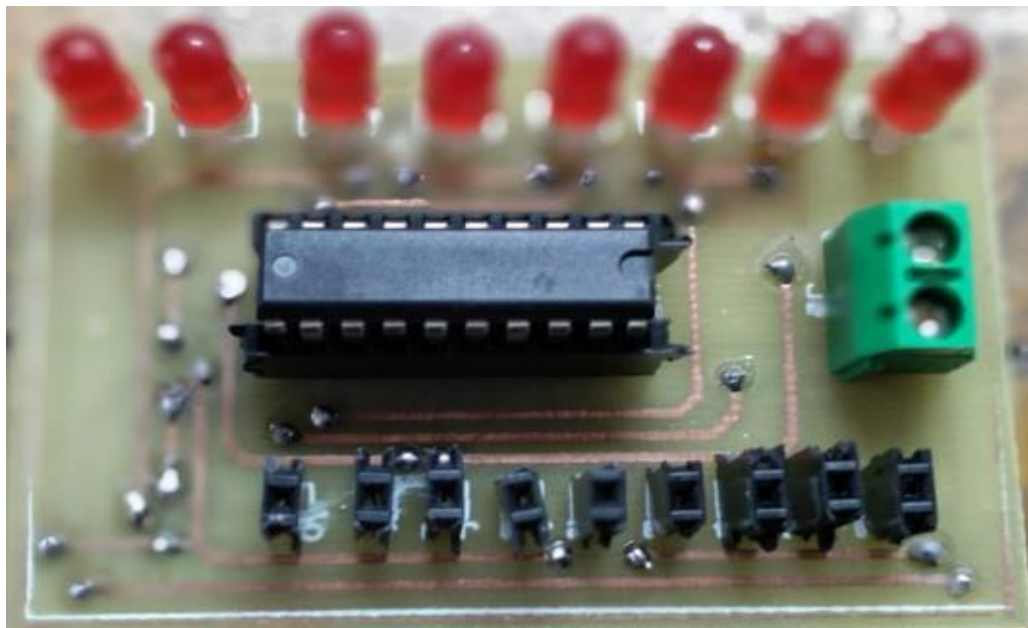
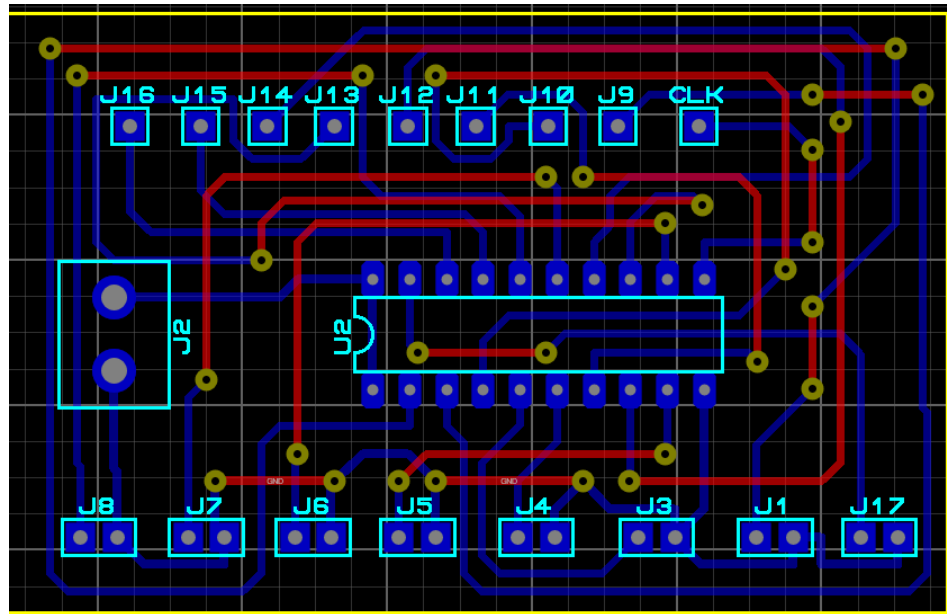
B Register



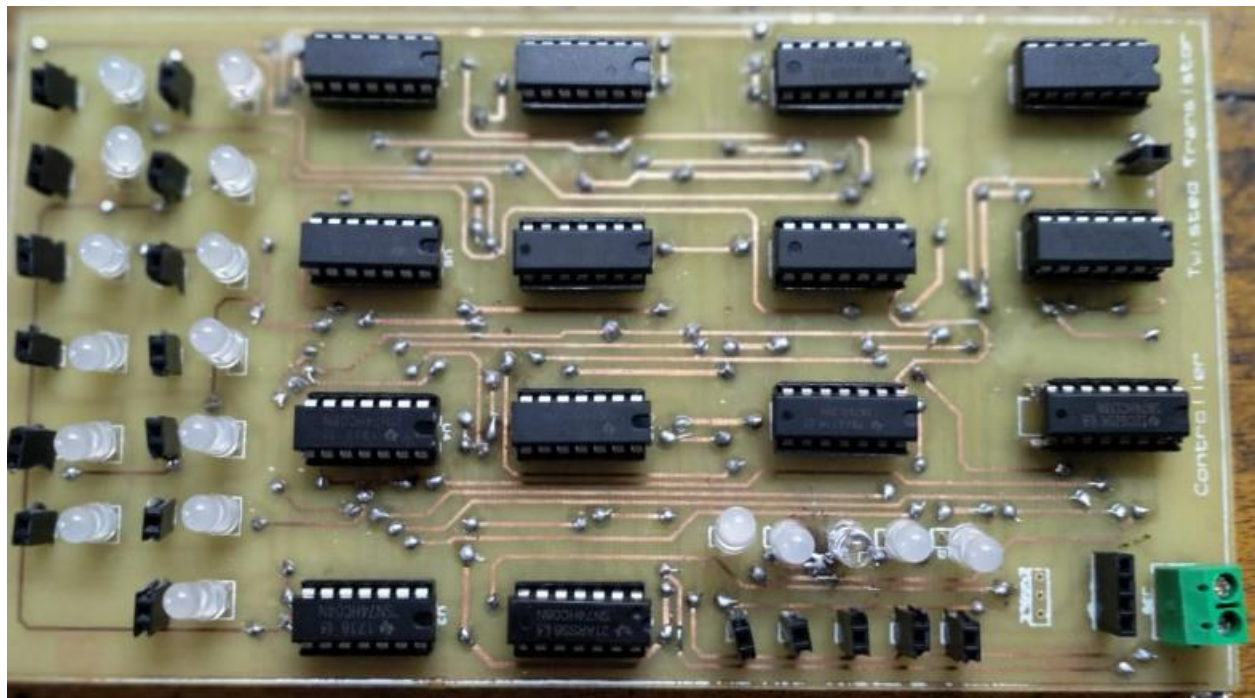
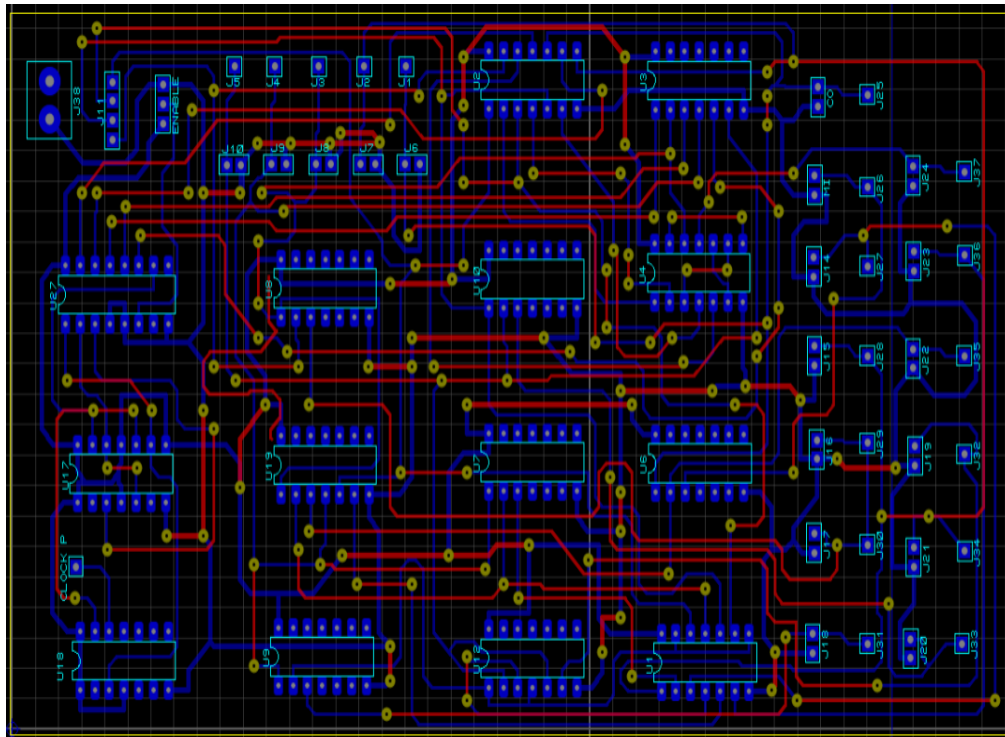
Instruction Register



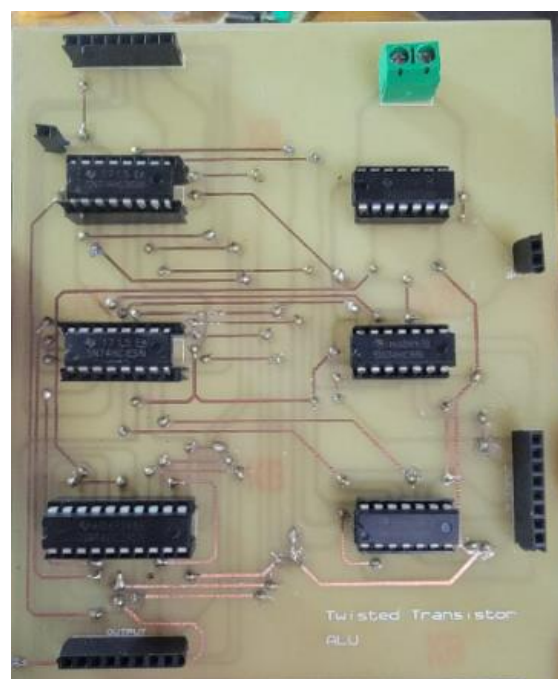
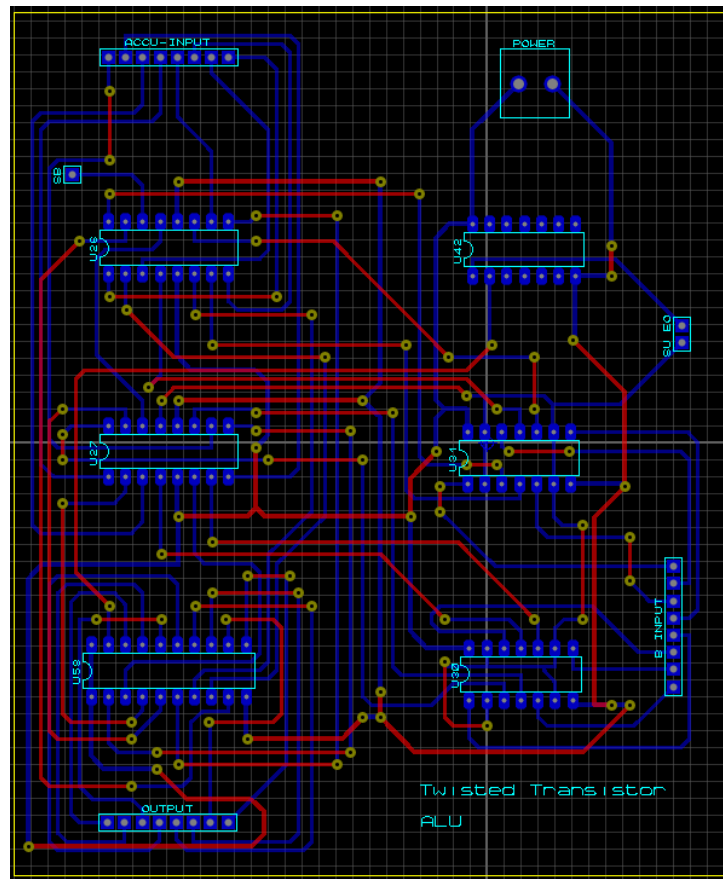
Output Register



Controller Sequencer



ALU



PCB Challenges:

As PCB designing concept, especially in Protues is new to us, we had to learn a lot through trial and error while making the PCB.

Not all components had PCB layout at first. So we had to replace those components with those which had PCB package. Then we had to think how to place them so we can place the components easily while soldering.

While designing PCB we had to place all the pads in bottom copper. We also set the trace-trace, pad-trace, pad-pad clearance. After auto routing many connections were missing. So, we had to connect them manually by manual routing. While connecting we faced some DRC and CRC errors due to misplacement. We also fixed that immediately.

During auto routing some connections remain missing, which we had to do manually. Sometimes adjacent vias get shorted during soldering. Then they had to be desoldered and soldered again with more precision.

We didn't give enough clearing in our first two PCBs, so it was difficult to solder them and they required much more precision and time.

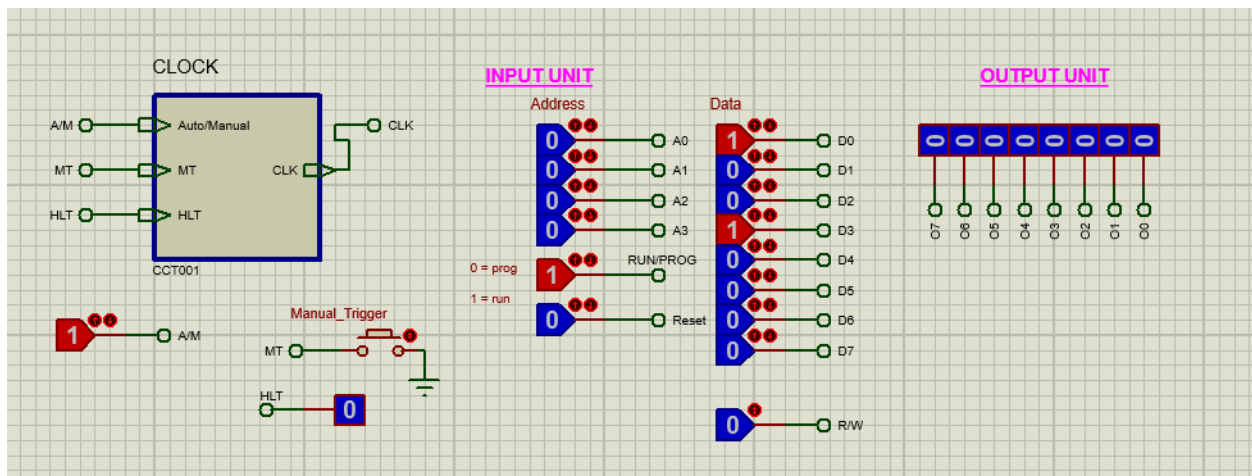
The distance between two adjacent pins in the IC bases and female headers is very small, so it was very challenging and time consuming to solder them.

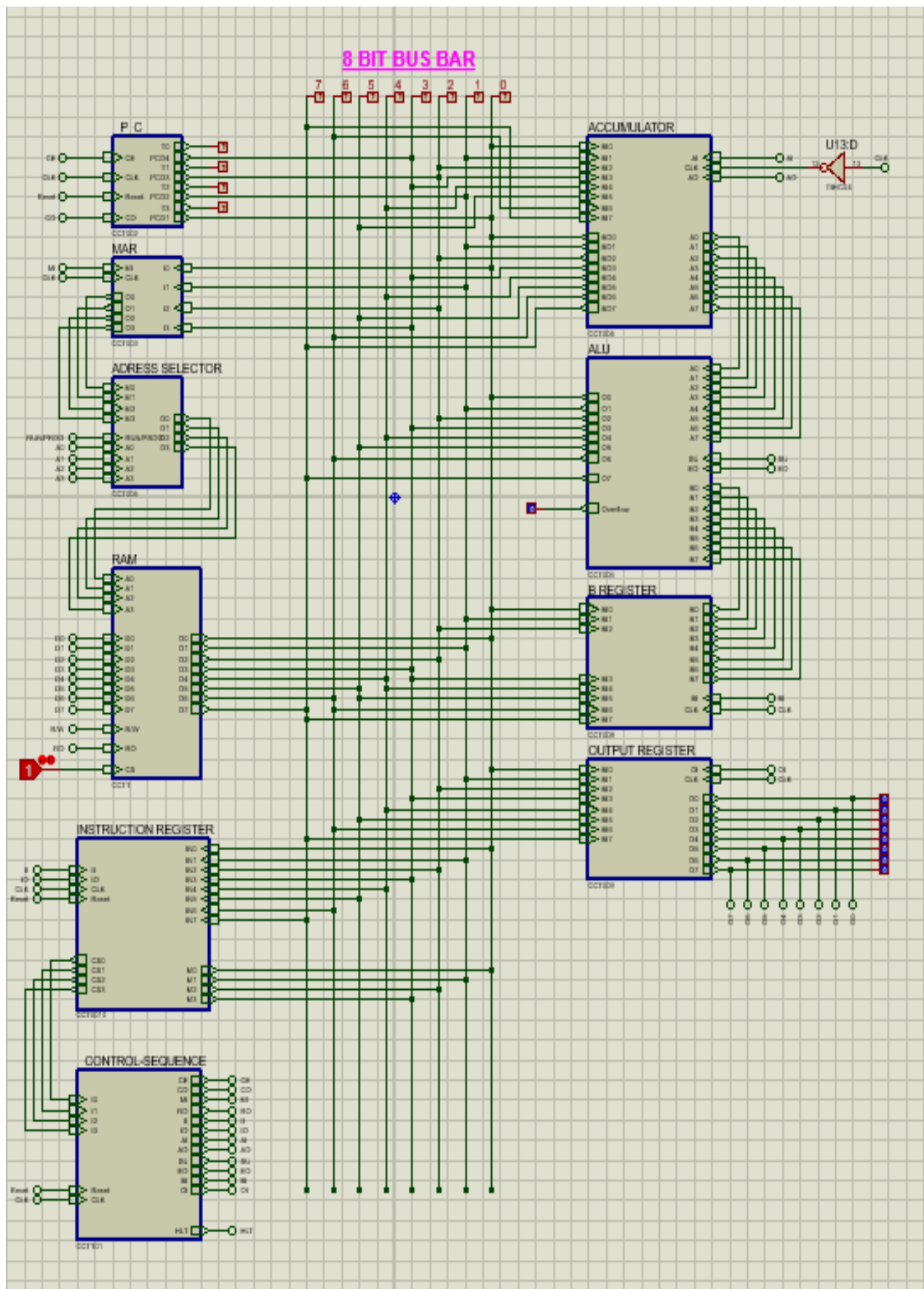
Although the PCB designs were flawless, due to faults in the "PCB printing", there were faulty connections in the PCB, which had to be debugged and connected manually.

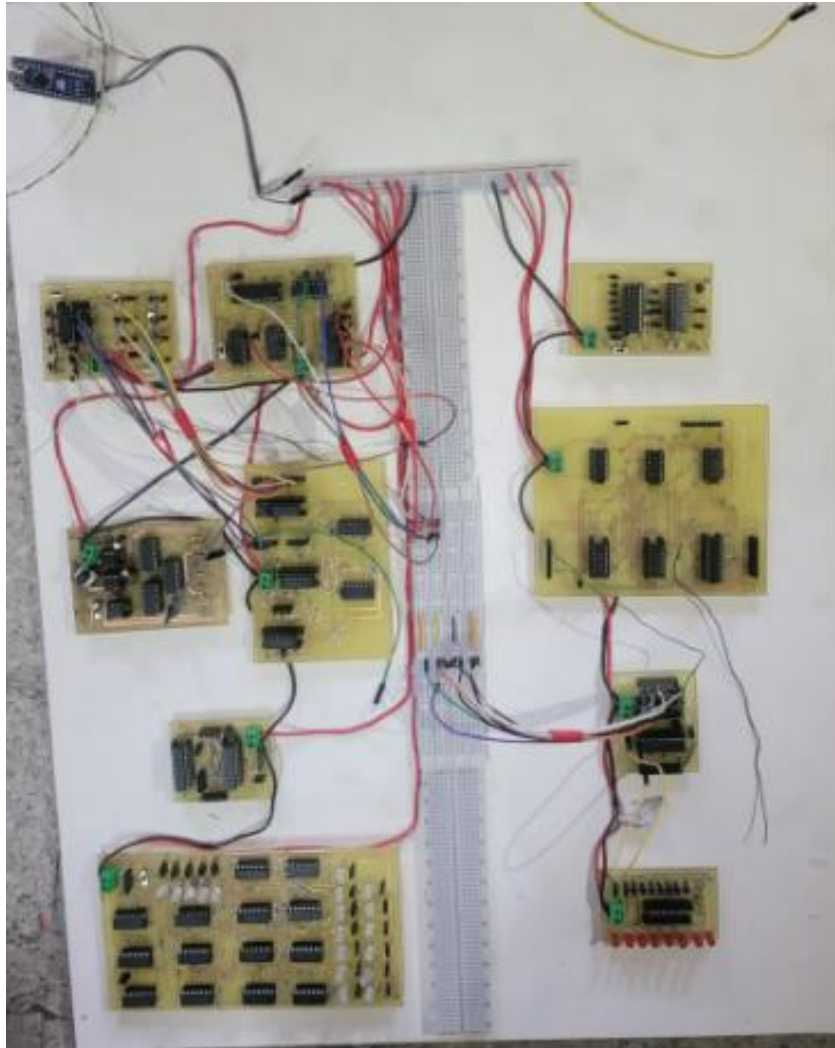
Compilation of all modules of SAP-1:

1. At first, we opened a new Proteus file for storing the compilation.
2. One by one, we copied and pasted each of the individual modules we constructed from phase A through phase D in the new Proteus file.
3. Then we connected the modules with each other.
4. Then we connected the modules with the central bus bar.
5. Debugged the compilation.
6. Did the final modifications and necessary formatting to ensure the compilation runs properly.

Proteus Structure:







Challenges faced while compilation:

1. While copying a new module's childsheet in the compilation, the previous existing module's childsheet was also getting replaced with the new module's childsheet.

To solve this problem, we had to change the subcircuit modules' string/ code which is situated below each of the subcircuits' in such a way that no two subcircuit possess the same string/code. While doing it we had to at first copy the childsheet in the clipboard to paste later on, because while changing the modules string/code the childsheet was getting deleted.

2. While connecting the different modules, we had a problem with ascertaining the MSB and LSB of the modules and the corresponding connections. Hence had to change and reconnect some connections of some modules after necessary debugging .

3. Missed some connections due to negligence, had to find them out by debugging.

SAP Testing

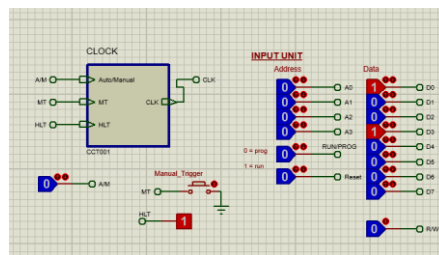
The primary functions of SAP 1 are **to add, to subtract** and **to load** 8-bit data.

Now we discuss some of these features in brief:

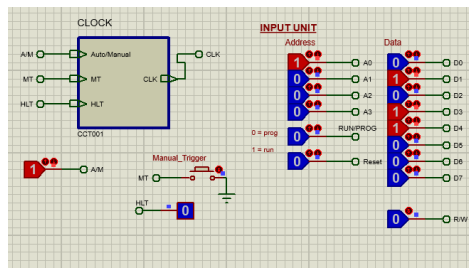
Addition:

- In Input Unit, we specify a location, and write 8-bit into the RAM, where first 4 bits are for Opcode and the other 4 bits are their address.
- In this way, we write two 8-bit data into the Ram in different locations.
- Here for writing data, first we load one 8 bit data. For loading purpose, the opcode is 0000.
- Then we input the data for addition. For addition purpose, the opcode is 0001.
- Then, we give the opcode input for OUT and HLT.
- Finally we sequentially give input two 8-bit binary data for addition purpose.
- Then we reset every module.
- When we give clock pulse, withing a second, the output shows the addition of the two 8-bit numbers.

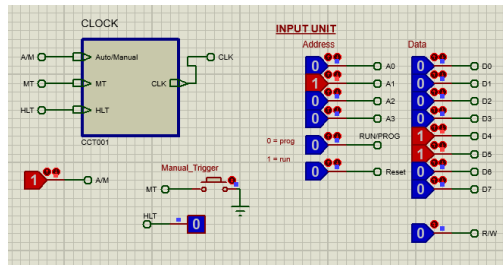
1. Load.



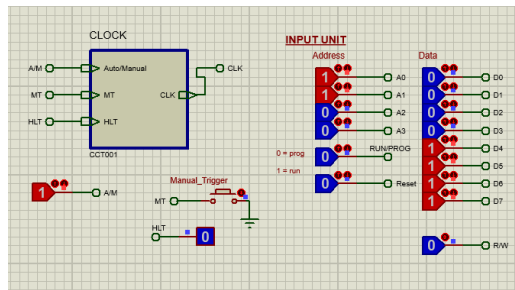
2. ADD.



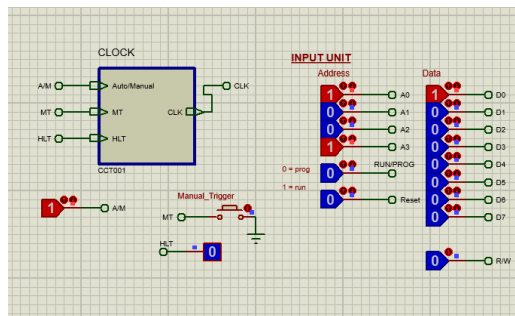
3. OUT.



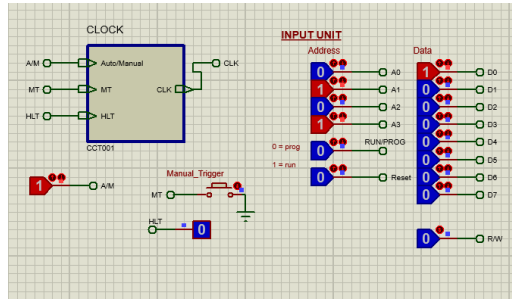
4.HLT.



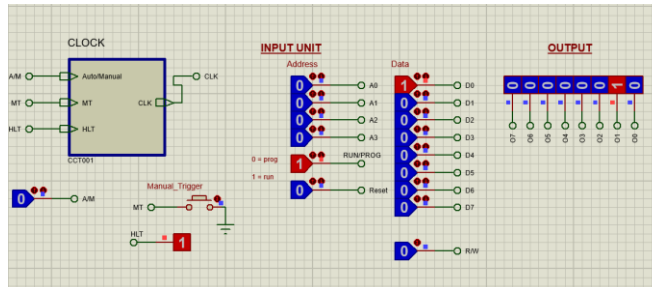
5. First 8-bit number. (00000001)



6. Second 8-bit number. (00000001)



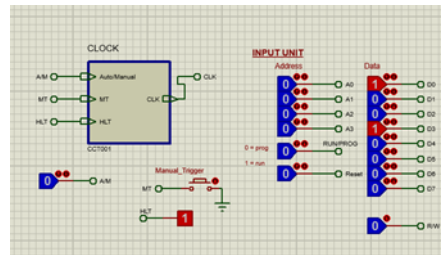
7. Output. (00000010)



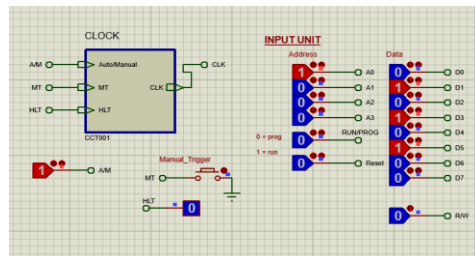
Subtraction:

- In Input Unit, we specify a location, and write 8-bit into the RAM, where first 4 bits are for Opcode and the other 4 bits are their address.
- In this way, we write two 8-bit data into the Ram in different locations.
- Here for writing data, first we load one 8 bit data. For loading purpose, the opcode is 0000.
- Then we input the data for subtraction. For subtraction purpose, the opcode is 0010.
- Then, we give the opcode input for OUT and HLT.
- Finally we sequentially give input two 8-bit binary data for subtraction purpose.
- Then we reset every module.
- When we give clock pulse, within a second, the output shows the subtraction of the two 8-bit numbers.

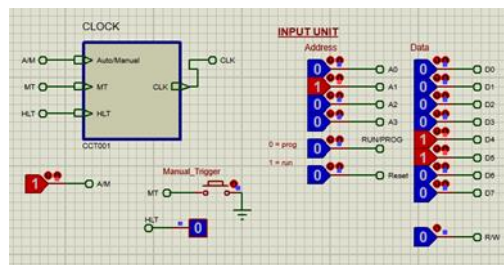
1. Load.



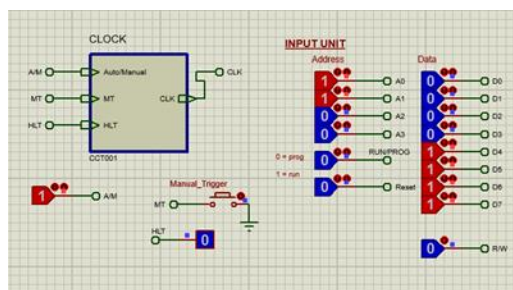
2. SUB.



3. OUT.



4. HLT.



5. First 8-bit number (00000011)

Project Expenditure

Cost of 10 PCB modules	3100
Cost of 8 breadboards	$120 \times 8 = 960$
2 jumper board sets	$200 \times 2 = 400$
IC	2500 (approx.)
IC base and female header	250
Soldering iron	600
Resistors and Capacitors	450
Jumper wires	250
Transportation cost for collection of different components	1200
Boards and Adhesive guns	300
Project Report Printing	500
Miscellaneous	1000
Total	11,510/-

Discussion:

In this project we built SAP-1(Simple As Possible-1).It was vast project. It included both hardware and software implementation. We did the software implementation in Proteus. Then at first, we did the hardware implementation on breadboard. Afterwards we created PCB layouts of the modules in proteus and printed them, soldered them and finally arranged and connected the whole SAP on a hardboard.

It was a tough project, but we learned a lot . It improved both our software and hardware skills with respect to materializing an idea into reality.

THE END